

THE UNIVERSITY OF CHICAGO

SCALING CPU USAGE IN GENOMICS WORKFLOWS

A DISSERTATION SUBMITTED TO  
THE FACULTY OF THE DIVISION OF THE PHYSICAL SCIENCES  
IN CANDIDACY FOR THE DEGREE OF  
DOCTOR OF PHILOSOPHY

DEPARTMENT OF COMPUTER SCIENCE

BY  
MARTIN L. PUTRA

CHICAGO, ILLINOIS

AUGUST 2026

Copyright © 2026 by Martin L. Putra  
All Rights Reserved

*Dedicated to my parents (Bapak Belanta, Ibu Nini), my brother Endha, and my sister  
Glory in Bekasi, Indonesia.*

# TABLE OF CONTENTS

|                                                                                               |      |
|-----------------------------------------------------------------------------------------------|------|
| LIST OF FIGURES . . . . .                                                                     | vi   |
| LIST OF TABLES . . . . .                                                                      | viii |
| ACKNOWLEDGMENTS . . . . .                                                                     | ix   |
| ABSTRACT . . . . .                                                                            | xi   |
| 1 INTRODUCTION . . . . .                                                                      | 1    |
| 1.1 Thesis Statement . . . . .                                                                | 2    |
| 1.2 Scaling CPU Usage in Genomics Workflows . . . . .                                         | 3    |
| 1.2.1 Semi-Automatic Translator with CNT . . . . .                                            | 3    |
| 1.2.2 Dynamic CPU Allocation with GENOX . . . . .                                             | 4    |
| 1.3 Contributions . . . . .                                                                   | 5    |
| 1.4 Thesis Organization . . . . .                                                             | 6    |
| 2 BACKGROUND . . . . .                                                                        | 7    |
| 2.1 Workflow Languages . . . . .                                                              | 7    |
| 2.2 Genomics Workflow Execution . . . . .                                                     | 8    |
| 2.3 CPU Quota Allocation in Production Practice . . . . .                                     | 11   |
| 3 CNT: SEMI-AUTOMATIC TRANSLATION FROM CWL TO NEXTFLOW FOR<br>GENOMIC WORKFLOWS . . . . .     | 12   |
| 3.1 Overview . . . . .                                                                        | 12   |
| 3.2 Design of CNT . . . . .                                                                   | 14   |
| 3.2.1 Tool-Level Translation . . . . .                                                        | 15   |
| 3.2.2 Graph-Dependency Analysis . . . . .                                                     | 17   |
| 3.2.3 Correctness Check . . . . .                                                             | 19   |
| 3.3 Manual Translation . . . . .                                                              | 20   |
| 3.3.1 Ensuring Syntax Conformance . . . . .                                                   | 20   |
| 3.4 Evaluation of CNT . . . . .                                                               | 21   |
| 3.4.1 Translation MD5 Similarity . . . . .                                                    | 22   |
| 3.4.2 Translation Coverage and Total Lines of Code (LOC) . . . . .                            | 23   |
| 3.4.3 Performance Gain by Adopting CNT . . . . .                                              | 24   |
| 3.5 Conclusion . . . . .                                                                      | 25   |
| 4 GENOX: EXPLOITING THREAD SIGNALS FOR DYNAMIC CPU SCALING IN<br>GENOMICS WORKFLOWS . . . . . | 26   |
| 4.1 Overview . . . . .                                                                        | 26   |
| 4.2 Challenges and Measurement . . . . .                                                      | 30   |
| 4.2.1 Input-dependent Performance Variability (C1) . . . . .                                  | 30   |
| 4.2.2 Contention-induced Slowdown (C2) . . . . .                                              | 33   |

|       |                                                    |    |
|-------|----------------------------------------------------|----|
| 4.2.3 | Dynamic Workflow Branching (C3)                    | 35 |
| 4.2.4 | Measurement Summary and Design Insights            | 36 |
| 4.3   | GENOX Design                                       | 36 |
| 4.3.1 | GENOX Architecture Overview                        | 36 |
| 4.3.2 | Thread-based Scaler for Responsiveness             | 37 |
| 4.3.3 | Fine-grained Thread Monitor for Robustness         | 39 |
| 4.3.4 | Container-native Controller for Adaptiveness       | 40 |
| 4.4   | Evaluation of GENOX                                | 41 |
| 4.4.1 | Evaluation Methodology                             | 42 |
| 4.4.2 | Micro-benchmarks (Case Studies)                    | 43 |
| 4.4.3 | Application-level Performance                      | 46 |
| 4.4.4 | Workflow-level Performance                         | 48 |
| 4.4.5 | Performance Projection on Increasing Input Sizes   | 49 |
| 4.4.6 | Performance on Varying Datasets                    | 51 |
| 4.5   | Discussion                                         | 53 |
| 4.6   | Conclusion                                         | 54 |
| 5     | RELATED WORK                                       | 56 |
| 5.1   | Automated Translation                              | 56 |
| 5.2   | CPU Quota Allocation                               | 57 |
| 6     | FUTURE WORKS                                       | 59 |
| 6.1   | Meta-Compiler for Language-Agnostic Workflows      | 59 |
| 6.2   | Cloud-Agnostic Execution                           | 60 |
| 6.3   | Minimizing Memory Waste in Containerized Execution | 60 |
| 6.4   | Reliable and Cost-Efficient Execution on Cloud     | 61 |
| 6.5   | Leveraging Domain-specific Characteristics         | 62 |
| 7     | CONCLUSION                                         | 63 |
|       | REFERENCES                                         | 65 |

## LIST OF FIGURES

|     |                                                                                                                                                                                                                                                                                                                                                                                                   |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | <b>Example of genomics workflow execution.</b> <i>A workflow DAG with input-dependent steps executes using given analysis and reference files. Some steps are skipped based on input, and each active app runs within a container.</i>                                                                                                                                                            | 9  |
| 3.1 | <b>Five steps of tool-level translation (§3.2.1).</b>                                                                                                                                                                                                                                                                                                                                             | 15 |
| 3.2 | <b>Coverage and LOC evaluation results. (§3.4.2)</b> <i>(a): x-axis: workflow types, y-axis: the percentage of coverage. (b): x-axis: workflow types, y-axis: the total LOC in thousands.</i>                                                                                                                                                                                                     | 23 |
| 4.1 | <b>Input-dependent performance variability.</b> <i>Genomics applications exhibit varying runtime growth with input size, up to 10× differences (4.1a) and shifting critical paths in DNA-Seq (4.1b) and RNA-Seq (4.1c).</i>                                                                                                                                                                       | 27 |
| 4.2 | <b>Critical path shifts with input size increase.</b> <i>‘Top’: Homogeneous scaling maintains the same critical path (as input size doubles). ‘Bottom’: Heterogeneous scaling causes the critical path to shift between applications (from blue to purple), complicating CPU quota allocation.</i>                                                                                                | 31 |
| 4.3 | <b>Inflatable parallelism (‘Phase B’) in BWA execution.</b> <i>Thread count dynamics across different phases for two configurations (16 and 96 threads). ‘Phase A:’ I/O-driven spikes, ‘Phase B:’ thread counts surge proportionally to user requests, ‘Phase C:’ a stable thread levels until completion. Phase B reveals configuration’s impact on resource consumption and execution time.</i> | 32 |
| 4.4 | <b>Stall heterogeneity across genomics applications.</b> <i>Distribution of CPU and I/O stall percentages showing diverse resource bottlenecks. Some applications are predominantly CPU-bound while others are I/O-bound, complicating utilization-based allocation under contention.</i>                                                                                                         | 33 |
| 4.5 | <b>Millisecond-scale idle burst dynamics in GATK.</b> <i>(a): CPU utilization shows large volatility over time. (b): fine-grained oscillations from alternating compute and I/O phases (b).</i>                                                                                                                                                                                                   | 34 |
| 4.6 | <b>GenoX architecture.</b> <i>GENOX operates alongside the workflow engine as a sidecar: its monitor collects thread counts, the scaler computes proportional CPU quotas, and quotas are enforced via Linux cgroups and the kernel’s CFS scheduler while applications run inside containers.</i>                                                                                                  | 37 |
| 4.7 | <b>Thread-level dynamics in genomics applications.</b> <i>In BWA, STAR, and GATK, thread activity shifts sharply across phases, exposing bursts of parallelism (inflatable parallelism) that utilization signals fail to capture.</i>                                                                                                                                                             | 39 |
| 4.8 | <b>Coarse-grained vs fine-grained monitoring.</b> <i>Utilization fluctuates with millisecond-scale idle bursts, while thread counts remain stable, enabling GENOX to base quota decisions on reliable signals.</i>                                                                                                                                                                                | 40 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.9  | <b>BWA case study results.</b> ‘(a) CPU utilization:’ Baselines lag behind bursts while GENOX immediately detects thread inflation. ‘(b) Quota allocation over time:’ GENOX scales up/down promptly with thread count whereas baselines react after demand shifts. ‘(c) Completion times:’ GENOX achieves the shortest runtime by eliminating/minimizing throttling penalties, while baselines finish later due to delayed reactions. . . . .                      | 44 |
| 4.10 | <b>STAR case study results.</b> (a) ‘CPU utilization:’ Baselines misinterpret idle bursts as reduced demand while GENOX tracks thread counts to maintain accurate allocation. (b) ‘Quota allocation over time:’ GENOX responds immediately to thread changes, whereas baselines exhibit delayed reactions. (c) ‘Completion times:’ GENOX achieves fastest runtime through prompt response and robustness to idle bursts. . . . .                                   | 44 |
| 4.11 | <b>BamSorMaDup case study results.</b> ‘(a) CPU utilization:’ UBaselines fluctuate due to I/O stalls while GENOX maintains stability matching the constant 16-thread profile. ‘(b) Quota allocation over time:’ GENOX consistently assigns one core per thread whereas baselines make unstable adjustments. ‘(c) Completion times:’ All policies finish similarly but GENOX avoids resource waste, demonstrating efficient adaptation to steady workloads. . . . . | 45 |
| 4.12 | <b>Execution times (4.12a) and straggler slowdown (4.12b) of DNA-Seq under contention.</b> ‘(a) Execution time:’ GENOX consistently reduces completion times across all applications, whereas baselines show inconsistent performance. ‘(b) Straggler slowdown:’ GENOX prevents performance degradation from stragglers, whereas baselines suffer severe delays (e.g., 200% slowdown for BWA under SHOWAR and 43% under Autopilot.) . . . . .                      | 46 |
| 4.13 | <b>Per-application start and finish times in DNA-Seq.</b> Baseline policies prolong key stages (e.g., P-OxO (Picard-CollectOxoGMetrics) by up to 26%), delaying downstream tasks and inflating overall runtime, while GENOX keeps stage completion times shorter and more balanced. . . . .                                                                                                                                                                        | 48 |
| 4.14 | <b>Per-application start and finish times in RNA-Seq.</b> Baselines extend long-running stages such as STAR (by 19% under SHOWAR), which cascades into delays for dependent tasks. GENOX reduces such stage-level bottlenecks, sustaining faster progression at the workflow level. . . . .                                                                                                                                                                        | 49 |
| 4.15 | <b>Execution time against increasing input size on DNA-Seq (4.15a) and RNA-Seq (4.15b)</b> ‘(a) DNA-Seq’ (BWA): GENOX maintains an average of 10-26% speedups on each input size compared to other policies, with an overall average speedup of 16.3%. ‘(b) RNA-Seq’ (STAR): GENOX achieves an average of 8-35% speedups on each input size, with an overall average speedup of 23%. . . . .                                                                       | 50 |
| 4.16 | <b>Execution time on various public DNA (4.16a) and RNA (4.16b) datasets.</b> ‘(a) DNA Datasets:’ GENOX achieves an average of 3.2-24.1% speedups per dataset, with an overall average of 11% across all DNA datasets. ‘(b) RNA Datasets:’ GENOX achieves an average of 8.3-35.6% speedups per dataset, with an overall average of 23% across all RNA datasets. . . . .                                                                                            | 52 |

## LIST OF TABLES

|     |                                                                                                                                                                                                                                                                         |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | <b>Summary of evaluation results (§3.4).</b>                                                                                                                                                                                                                            | 13 |
| 3.2 | <b>Summary of value substitution (§3.2.1).</b> <i>E: elementary item that does not require processing, C: a collection that requires one before substitution.</i>                                                                                                       | 15 |
| 3.3 | <b>Translation MD5 similarity for automated-translation of CNT (§3.4.1).</b> <i>Column names "O", "OV", and "S" are shortnames for template placeholders described in §3.2.1. O: Output, OV: Optional Variable, S: Script.</i>                                          | 22 |
| 4.1 | <b>Contention-induced slowdowns across application pairs.</b> <i>Entries show normalized execution time; larger values indicate higher slowdown. Trimm.: Trimmomatic, P-DNA: Picard-DNA, P-RNA: Picard-RNA</i>                                                          | 34 |
| 4.2 | <b>CPU usage patterns of three representative genomics applications.</b> <i>Each application is selected to demonstrate a particular design choice of GENOX. i.e., responsiveness (BWA), responsiveness and robustness (STAR), and adaptivity (BamSorMaDup).</i>        | 43 |
| 4.3 | <b>End-to-end execution times (sec.) for DNA-Seq and RNA-Seq workflows.</b> <i>GENOX consistently achieves the shortest runtimes, speeding up DNA-Seq by 12 – 18% and RNA-Seq by 19 – 25% compared to baselines.</i>                                                    | 47 |
| 4.4 | <b>Details of DNA and RNA datasets.</b> <i>The collected datasets have different input characteristics, as represented by diverse experimental strategy, sequencing hardware, GC content, and total file size.</i>                                                      | 51 |
| 4.5 | <b>Execution times of DNA-Seq and RNA-Seq workflows without contention.</b> <i>In resource-rich environments, GENOX performs nearly identically to Linux's NoLimit policy, with only about 1% difference.</i>                                                           | 53 |
| 5.1 | <b>GenoX compared with prior CPU allocation systems for genomics workloads.</b> <i>In the context of genomics workload execution, existing systems achieve only a subset of properties, while GENOX uniquely provides responsiveness, robustness, and adaptiveness.</i> | 58 |



## ACKNOWLEDGMENTS

Praise be to God for His blessings. My PhD journey has been made possible through the help and guidance of so many individuals across places. I extend my heartfelt gratitude to them here.

First and utmost, I am beyond grateful for my PhD advisors, Robert L. Grossman and Haryadi S. Gunawi. Both have given me irreplaceable guidance and mentorship throughout years of my PhD journey. I remember the many times where I failed to meet their expectations, yet they keep believing in me and supporting me. Their unwavering trust and steadfast support kept me going until I'm finally able to finish this dissertation. I can not express how thankful I am to have them as my advisors.

Second, I am also beyond grateful for my committee member In Kee Kim from the University of Georgia. In Kee has been closely mentoring me since my second year, and is often the first person I talk to when I need to exchange thoughts and ideas about my research. Like my advisors, he also has given me unwavering trust, support, and guidance through numerous setbacks and difficult moments. I am thankful for everything he has done for me throughout my journey.

To the members of Grossman Lab: Michael Hao Tong, Renyu Zhang, Steven Song, Matthew West, Michael Lukowski, and Summer Ann. The members of UCARE Lab: Huaicheng Li, Mingzhe Hao, Cesar Stuardo, Daniar Kurniawan, Ray Andrew Sinurat, Roy Huang, Ruidan Li, Maharani Irawan, and William Nixon. Thank you for the mentorship and companionship as I faced the ups and downs of PhD life. Our group lunches are always enjoyable and insightful.

To my friends in the University of Chicago: Seyed Esmaeili, Andrzej Kaczmarczyk, Emily Wenger, Antares Chen, Byron Zhong, Bogdan Alexandru Stoica, Samuel Lee, Calvin Raab, Clara Kyung, Jia Ahn Pan, etc. Thank you for all the conversations and fun times we had together. You guys made living in a foreign country bearable to me.

To the Christian communities in campus and Hyde Park: the Living Hope Church, Christ Church Chicago, GCF, Lumen Christi Institute, and Bond Chapel Prayer Group. Our activities nourish me spiritually and allow me to grow as a person. I will miss our time together.

Lastly, I would like to express my gratitude to my family. I would not be able to walk this far without the love and support they have given me. Thank you for believing in me, for being there when I was down, and for accepting me despite my flaws. I dedicate this dissertation to you.

# ABSTRACT

DAG-structured genomics workflows process gigabytes to terabytes of sequencing data in shared clusters, where efficient CPU usage is critical in resource management. Inefficient CPU usage can significantly delay workflow executions and increase computational costs. Two possible reasons for this issue are limited degree of parallelism and suboptimal CPU allocation during execution. This dissertation presents systems which we developed to scale the CPU usage of genomics workflows.

In the first work, we address the issue of limited degree of parallelism by facilitating easier transition between workflow engines. Genomics workflows execution in production uses workflow engines in order to provide reproducibility, data provenance, task scheduling, and other desirable properties. However, the degree of task parallelism that can be achieved during execution vary across various engines. Some engines can only run workflow tasks serially, like cwltool, while others, like Nextflow, support more sophisticated parallel task execution. Transitioning to Nextflow poses a significant challenge due to the differences in programming models, scripting language compatibilities, and the prerequisite for in-depth knowledge between Nextflow language and CWL, the workflow language of cwltool. We develop **CNT**, a semi-automatic translator which allows easier migration of CWL workflows to Nextflow. This allows us to execute workflows using the Nextflow engine, which provides better task parallelism and consequently results in higher CPU usage. We evaluate the performance improvements achieved by using Nextflow workflows translated by CNT vs. running CWL on cwltool.

In the second work, we address CPU allocation during workflow execution. Existing CPU allocation approaches are often insufficient for genomic workflows due to their unique challenges: input-dependent performance variability, contention-induced interference, and dynamic workflow branching. We first characterize production-level workflows for processing DNA and RNA sequencing data, identifying three critical characteristics: inflatable paral-

lelism (thread surges up to  $96\times$ ), stall heterogeneity (up to  $2\times$  slowdowns) and millisecond-scale idle bursts that make utilization often unreliable. These findings confirm that effective CPU management should satisfy three requirements: 1) responsiveness to thread bursts, 2) robustness under contention, and 3) adaptiveness to workflow dynamics. Motivated by our characterization study, we developed GENOX, an automated CPU quota management system that leverages thread counts as its primary signal for CPU scaling, rather than traditional metrics like utilization. Operating at 5ms intervals through standard Linux cgroups, GENOX responds immediately to changes in CPU demand phases while remaining stable under contention. Evaluation results show that GENOX outperforms state-of-the-art baselines, as it reduces application execution time by up to 50%, eliminates 200% straggler slowdowns, and improves workflow completion by 21%. These evaluation results confirm that thread-based scaling effectively addresses CPU allocation challenges in genomics workflows.

# CHAPTER 1

## INTRODUCTION

A large-scale genomics data processing system requires tremendous amount of resources to operate. For example, a report on one of the largest genomics cloud platforms [1] shows that it consists of hundreds of compute nodes, containing thousands of cores, tens of terabytes of memory, and nearly a petabyte of local storage. Genomics workload is especially computationally demanding. The system processes hundreds of thousands of workflows every year, many of which contain CPU-intensive steps, and in total consumes 35 million core hours annually. Thus, it is necessary to efficiently utilize CPU resources in order to maintain optimal system throughput.

However, achieving consistently high CPU utilization is difficult. A system may have suboptimal CPU utilization due to the limitations of its workflow engine. Existing workflow engines have varying degrees of task parallelism support, and a system might need to migrate its legacy workflows in order to use a newer workflow engine with better parallelism support.

Yet migrating legacy production workflows from one language to another is challenging due to the required time and in-depth domain-specific expertise. Production workflows are often very complex, as each encompasses various logic to ensure robustness and high performance across multiple environments, in addition to its main data transformation process. As a result, an accurate translation that processes data uniformly compared to the original workflow often requires long person-hours of development and testing. Moreover, most production workflows use advanced workflow language features and patterns. This makes translation require deep expertise in both the source and target languages.

Suboptimal CPU utilization can also happen due to inefficient allocation. Allocation needs to avoid overallocation, which is wasteful, and underallocation, which can cause a slowdown due to insufficient resources. Yet, the computational demand of genomics workloads is not static, and the right allocation approach needs to cope with dynamic changes

throughout execution.

Current CPU allocation approaches, which are designed for other domains, are either inadequate for genomics workloads or difficult to implement. They are unable to react promptly to the dynamic demands of genomics workloads. Some require historical data for predictive model training that may be infeasible to obtain considering the diversity and performance variability of genomics applications. They can also demand modifications at the application/kernel-level which are difficult to maintain in production system.

Despite the scale and importance of genomics data processing, works that specifically address its systems challenges remain scarce. This compels us to embark on a journey of exploration to strengthen genomics data processing systems, specifically its CPU usage aspect.

## 1.1 Thesis Statement

In this dissertation, we seek to answer the following question: *How can we increase the CPU usage of genomics workflows in order to ensure optimal systems throughput?* Specifically, we make the following thesis statement:

**Scaling the CPU usage of genomics workflows requires a high degree of task parallelism and an efficient CPU allocation, which can be achieved through the use of an optimized workflow engine and a dynamic runtime system that exploits unique genomics workload characteristics.**

This dissertation consists of two main parts, which collectively support the thesis statement. In the first part (Chapter 3), we develop a semi-automated workflow code translator that allows easier migration to a workflow engine that supports advanced parallelization. In the second part (Chapter 4), we discuss the unique characteristics of genomics workloads that make it challenging to allocate CPU efficiently, and proceed to propose a system that dynamically scales CPU usage. In summary, this dissertation proposes the following systems:

- **CNT**: A semi-automatic translator of CWL workflows [2] to Nextflow workflows [3] that supports both tool-level and workflow-level translation.
- **GenoX**: An automated CPU scaling system that exploits thread signals against unique genomics workload characteristics.

The rest of this chapter is organized as follows. In Section §1.2, we elaborate on how CNT and GENOX contribute to support our thesis statement. Following that, we summarize this dissertation’s contributions in Section §1.3. Finally, we present the organization of this dissertation in Section §1.4.

## 1.2 Scaling CPU Usage in Genomics Workflows

### 1.2.1 Semi-Automatic Translator with CNT

Many of the most popular open-source production-grade workflows are written in CWL. For example, GDC harmonization workflows [4], which have been used to process more than 2.5 PB of data, are written in CWL. Due to their quality and popularity, many other research groups are interested in processing their data uniformly using GDC’s harmonization workflows. Such uniform processing minimizes batch effects and enables better joint analysis across multiple datasets [5].

At the same time, with the rising popularity of Nextflow [3], there is significant interest from the bioinformatics and cancer research community to translate existing CWL workflows into Nextflow. Nextflow’s engine provides advanced support for task and data parallelism. Migrating from a workflow engine with a low degree of parallelism into Nextflow can yield a significant increase in CPU utilization and subsequently improve system throughput.

In the first part of this dissertation (Chapter 3), we introduce CNT, a semi-automatic translator that facilitates easier migration of a large codebase of production-grade CWL workflows into Nextflow. CNT’s contributions lie in its three-stages design: a tool-level trans-

lation, a workflow-level translation, and a correctness check. Each stage solves challenges that arise from CWL’s and Nextflow’s unique characteristics that we observe in production-grade workflows, such as CWLs’ heavy use of JavaScript and Nextflow’s dataflow operators and patterns.

We evaluate CNT using two of the longest-running GDC harmonization workflows, namely the DNA-Seq and RNA-Seq [4]. The results show that CNT can automatically translate up to 81% of CWL code to Nextflow with a correctness rate of up to 71%, thus significantly alleviating the time-consuming process of manual translation. Furthermore, when compared to running CWL on cwltool [6], the resulting Nextflow workflows achieve execution time reductions of 72% in RNA-Seq and 39% in DNA-Seq, along with an average increase in CPU utilization by 65% and 25.5%, respectively.

### 1.2.2 *Dynamic CPU Allocation with GENOX*

In the second part of this dissertation (Chapter 4), we introduce GENOX, our dynamic CPU allocation system for genomics workflows. GENOX is designed to solve specific genomics workload challenges that render existing static and dynamic allocation approaches from other domains ineffective. Our characterization study (§4.2) shows that, on top of the well-known challenge of heterogeneous resource demand among the variety of tools used in a workflow [7, 8], genomics workloads also exhibit unique challenges that further complicate efficient CPU allocation. Specifically, we observe three unique challenges: performance variability due to input characteristics, millisecond idle bursts due to contention-induced slowdown, and shifting resource demand due to workflow conditional branches.

We found that existing CPU allocation approaches, including utilization-based [9–11], ML-based predictors [12–14], and kernel-integrated approaches [15, 16], are all inadequate to face these challenges. Existing approaches either react with delays in their allocation, thus causing performance penalties, or are difficult to implement in production. In contrast,



GENOX is specifically developed to solve these unique challenges. GENOX’s contribution lies in a design that is grounded in insights from our characterization study. GENOX consists of three components: a thread-based scaler, fine-grained monitor, and container-native controller. Each component tackles the issue of responsiveness, robustness, and adaptiveness that is necessary to efficiently allocate CPU for genomics workloads.

We evaluate GENOX across multiple microbenchmarks and macrobenchmarks in production-like environments. Our evaluation shows that GENOX consistently outperforms state-of-the-art CPU allocation approaches, achieving up to 50% reduction in execution time and minimizing straggler slowdowns at the application-level. It also improves end-to-end workflow completion time by over 20%. Furthermore, GENOX integrates well with the genomics data processing stack – it does not require any modification at the application or kernel level, thus making it easy to deploy in a production system.

### 1.3 Contributions

In summary, this dissertation makes the following contributions:

- We develop the first semi-automatic translator from CWL to Nextflow. The tool helps increase a system’s CPU utilization by facilitating migration from workflow engines with low degree of parallelism to Nextflow engine, which natively supports advanced parallelism.
- We perform an in-depth characterization study of genomics workloads that are widely used in production. The characterization study uncovers unique challenges pertaining to CPU allocation, along with domain-specific insights for building an efficient allocation system.
- We design a dynamic, measurement-driven CPU allocation system that uses threads as a leading signal with fine-grained monitoring to detect true demand. In addition,

we present a non-invasive implementation that uses only the existing cgroup interface, thus easily integrating with production systems.

## 1.4 Thesis Organization

The rest of this dissertation is organized as follows. In Chapter 2, we first present background information on genomics workflows which are common for both of our solutions, before we delve into their details. Then, in Chapter 3, we present **CNT**, our solution for migrating production-grade CWL workflows to Nextflow. Chapter 4 presents our in-depth characterization study of genomics workloads and introduces **GENOX**, our solution for efficient CPU allocation. In Chapter 5, we describe works which are related to **CNT** and **GENOX**. Chapter 6 discusses various potential future works to strengthen genomics data processing systems. Finally, we conclude our dissertation in Chapter 7.

## CHAPTER 2

### BACKGROUND

Modern genomics workflows integrate dozens of containerized applications into pipelines that process terabytes of biological data. Workflows are written in workflow languages (*e.g.*, Nextflow [17], CWL [18]), workflow engines coordinate task execution (*e.g.*, cwltool [6], Nextflow, Cromwell [19]), while containers provide resource isolation [20–22]. We first briefly describe workflow languages and the differences between CWL and Nextflow. We then outline how genomics workflows operate in production environments and why CPU quota allocation is a key performance concern.

#### 2.1 Workflow Languages

Protocols for analyzing bioinformatics data are often represented as computational workflows. Many workflow languages have been developed to represent such workflows. These languages are designed to achieve different goals, such as portability and reproducibility (*e.g.*, CWL [23]), human-readability/writability (*e.g.*, WDL [24]), up to fine-grained control of dataflow (*e.g.*, Nextflow [17]). Some languages, such as CWL, are decoupled from an execution engine, while other languages, such as Nextflow, are closely integrated with the execution engine.

**Common Workflow Language (CWL).** To achieve portability and reproducibility in bioinformatics workflows, CWL aims to capture the necessary workflow information while remaining high-level enough to be executed across diverse computing environments. CWL uses two core classes of files to represent workflows: *CommandLineTool* and *Workflow*. The ‘CommandLineTool’ is the smallest building block of a workflow, describing at least a specific command-line application, its parameters, input data requirements, and the output it produces (the full list of supported information can be found in [2]). CommandLineTool can

also incorporate additional information including software containers, computing resource requirements, and file formats for both input and output data. The ‘Workflow’ class takes a CommandLineTool file as a single computational step and composes it into a whole computational workflow. Workflows can be nested, allowing a single step within a Workflow to be another Workflow file up to arbitrary depths. Furthermore, CWL allows the use of JavaScript code to manipulate output files, such as changing names and constructing/deconstructing more complex data structures.

**Nextflow.** Analogous to CWL, Nextflow also has two classes which represent the smallest workflow building block and a whole computational workflow: *Process* and *Workflow*. The ‘Process’ class consists of three parts: input, output, and script blocks. The input and output define the I/O data, whereas the script block allows developers to write command-line commands (*e.g.*, shell commands). Like CWL, the ‘Workflow’ class in Nextflow takes one or more Process and composes them into a whole final workflow. However, Nextflow employs Groovy as its scripting language instead of JavaScript. Furthermore, Nextflow is based on dataflow programming, requiring an in-depth understanding of this paradigm by developers [17]. It introduces many dataflow operators which, when used appropriately, can result in efficient use of advanced parallelism during workflow execution.

## 2.2 Genomics Workflow Execution

Workflow executions consists of several important components, including workflows, workflow engines, containers, and input files. Discussions around it also requires understanding of several important concepts. Here we describe the most relevant components and important concepts that are used throughout this dissertation. Fig. 2.1 illustrates how they interact within production environments.

**Workflow.** A genomics workflow consists of interdependent steps (*e.g.*, alignment [25], variant calling [26], annotation [27]) represented as a DAG [28], where nodes are containerized

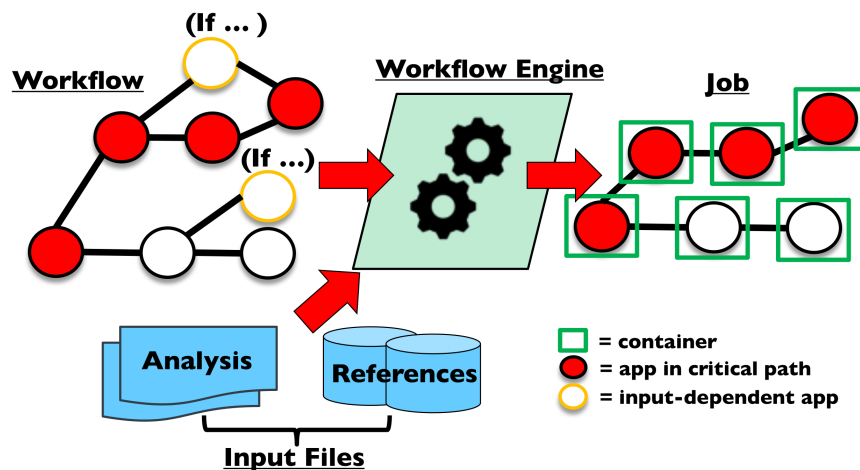


Figure 2.1: **Example of genomics workflow execution.** A workflow DAG with input-dependent steps executes using given analysis and reference files. Some steps are skipped based on input, and each active app runs within a container.

applications, and edges are file-based dependencies. Workflows are described using languages, *e.g.*, CWL, WDL [24], or Nextflow DSL. These languages support optional inputs and conditional branches, allowing the same workflow definition to activate different steps depending on the dataset.

**Job.** A job is a single workflow execution with a specific dataset. Optional inputs and conditional branches enable identical workflow definitions to produce different execution paths.

**Deployment.** Production workflows are executed by workflow engines (*e.g.*, Nextflow, Cromwell [19], CWLtool [6]) with container runtimes (Docker, Singularity [21,29]). Workflow engines schedule tasks and manage dependencies, while containers enforce CPU and memory quotas via Linux cgroups, which we leverage for dynamic and runtime CPU allocation.

**Input.** Genomics workflows process multiple types of input data. One common type is reference files (*e.g.*, the human genome reference [30]), serving as a baseline for analysis. Another common type is data files that contain DNA or RNA sequencing data. These sequencing data are created using various experimental strategies (*e.g.* WGS/WXS/Chip-Seq for DNA, RNA-Seq/RIP-Seq for RNA) and sequencing hardware (e.g. Illumina, Ion Tor-

rent, PacBio) [31], and can have different amount of GC content which is correlated with sequencing quality [32]. These sequencing files are typically accompanied by metadata, such as quality scores and processing parameters. The characteristics of input data have a direct impact on a genomics workflow’s execution time. For example, the input data size determines the required computational resources (*e.g.*, the number of CPU cores and memory allocation size), and the input format (*e.g.*, FASTQ or BAM) affects the I/O patterns observed during workflow execution. In addition, the presence of optional inputs enables conditional execution paths within the workflow. Specifically, input-dependent branching, due to the optional inputs, can make identical workflows to execute different subsets of applications depending on the input datasets [22, 28]. This behavior introduces performance variability that complicates performance prediction. We further discuss this challenge in §4.2.1.

**Critical path.** A workflow’s completion time depends on its critical path, which is the longest chain of dependent steps in the DAG [33]. While identifying critical paths is straightforward in controlled settings, production environments with input-dependent dynamic branching make this substantially more complex. The same workflow definition may activate different steps for different inputs, resulting in different critical paths. This variability complicates performance prediction and makes CPU quota allocation challenging in practice.

**Alignment workflow.** Alignment matches DNA or RNA sequencing reads to a reference genome [34]. While the core algorithm is string matching, production workflows add extensive preprocessing, quality control, and metrics collection that significantly impact runtime. These alignment jobs are among the most resource-intensive tasks in genomics, often consuming tens of CPU cores and hundreds of gigabytes of memory for days or weeks. Alignment jobs typically run in groups of 50 – 100, quickly saturating cluster resources with thousands of cores and huge memory demand. This makes optimizing alignment jobs critical for cost-efficient large-scale genomics processing and drives the need for more effective CPU quota

allocation strategies.

## 2.3 CPU Quota Allocation in Production Practice

Genomics workflows execute on shared clusters where multiple workloads compete for CPU resources. Current allocation schemes struggle with both over-allocation (wasting resources) and under-allocation (slowing down executions due to insufficient resources) problems, compounded by genomics workflows’ runtime variability. Existing systems employ either static allocation, where resources are fixed at submission time (*e.g.*, SLURM [35], PBS [36, 37]), or dynamic allocation, where CPU quotas are adjusted based on utilization (*e.g.*, Kubernetes HPA [38], YARN [39]). However, both fail to handle genomics-specific challenges. For example, static allocation cannot adapt to runtime variations, and dynamic approaches suffer from inherent lag during demand bursts. We further discuss these challenges and our solution in §4.

## CHAPTER 3

# CNT: SEMI-AUTOMATIC TRANSLATION FROM CWL TO NEXTFLOW FOR GENOMIC WORKFLOWS

### 3.1 Overview

Workflow languages are widely used when production systems analyze biomedical data since they provide consistent processing of batches of data, reproducible processing of a particular dataset, restart and reentry, data provenance, and other desirable properties, such as the ability to easily share bioinformatics pipelines with others. The popularity of particular workflow languages and workflow managers varies by subfield, data type, computing environment, and over time. For this reason, a wide variety of workflow languages and workflow managers are used in bioinformatics. For example, the survey [40] mentions over 150 workflow managers in use or in production.

Given the variety of workflow languages, it is desirable to have reliable tools that translate a workflow from one workflow language to another. For example, the Genomic Data Commons (GDC) [41] has developed 11 production CWL workflows [5] that are used for processing all the data submitted to the GDC. Given the popularity of the GDC and the amount of data it processes, others are interested in using the same workflows so that the data may be processed uniformly [5]. On the other hand, different groups may want to use different workflow managers.

With the growing popularity of Nextflow [3], there is significant interest within the bioinformatics and cancer genomics community in having a tool that translates CWL into Nextflow, as they have developed numerous CWL pipelines. While converting a CWL workflow to Nextflow manually is possible, this approach is exhaustively time-consuming and demands in-depth domain-specific expertise. For example, based on our experience, translating a production-grade pipeline, such as the GDC DNA-Seq workflow from CWL to Nextflow,



|                          | DNA-Seq | RNA-Seq |
|--------------------------|---------|---------|
| Speedup over cwltool [6] | 21-39%  | 33-72%  |
| CPU Utilization Increase | 18-33%  | 45-85%  |
| Translation Coverage     | 73%     | 81%     |

Table 3.1: **Summary of evaluation results (§3.4).**

takes approximately 160 person-hours. This estimate does not include testing and integration time.

Translating a CWL workflow into Nextflow presents several significant challenges. First, Nextflow follows a dataflow programming model, whereas CWL is declarative. This requires translation efforts to ensure the correctness of the dataflow model in Nextflow using various Nextflow operators. In contrast, CWL often relies on the workflow engine to support dataflow correctness, resulting in the explicit dataflow being commonly overlooked in workflow descriptions. Second, existing CWL workflows heavily utilize JavaScript, which must be translated into Nextflow’s scripting counterpart (*e.g.*, Groovy). This implies that the workflow translation involves not only moving from CWL to Nextflow but also transitioning from JavaScript to another language. Finally, to achieve the desired performance, developers must be familiar with advanced task parallelism supported in Nextflow and apply the appropriate Nextflow operators.

To address these challenges and maximize the benefits of adopting Nextflow, we developed **CNT**, which is, to the best of our knowledge, *the first translator* semi-automatically converting CWL workflows into Nextflow. **CNT** operates in three stages: a tool-level translation, graph dependency analysis, and a correctness check. In each stage, we tackle specific challenges. The tool-level translation converts CWL’s smallest workflow components, CommandLineTool files, into Nextflow’s Processes through a five-step approach based on essential CommandLineTools fields. Subsequently, the graph dependency analysis organizes these components into a complete workflow, addressing critical ordering challenges related to input/output variables in a Process, the sequence of Processes in a subworkflow, and

the arrangement of subworkflows in the final workflow. Finally, **CNT** ensures correctness by performing typecasting that verifies the use of appropriate data types for Nextflow processes and operators. Moreover, while **CNT** can automatically translate a significant portion (*e.g.*, 73% – 81%) of CWL code to Nextflow, certain code segments *may* require manual intervention, such as developer-written JavaScript codes. To enhance the translation process, **CNT** identifies three JavaScript code patterns that guide developers during the translation.

To evaluate **CNT**’s performance, we used **CNT** to translate the GDC DNA-Seq and RNA-Seq pipelines into Nextflow. Our evaluation first focuses on measuring translation coverage and correctness. The results show that **CNT** can automatically translate up to 81% of CWL code to Nextflow with a correctness rate of up to 71%. Subsequently, we evaluate the performance improvements achieved by using Nextflow workflows translated by **CNT** vs. running CWL on cwltool [6], the reference implementation of CWL. The results reveal significant speedups, with execution time reductions of 72% in RNA-Seq and 39% in DNA-Seq, along with an average increase in CPU utilization by 65% and 25.5%, respectively.

## 3.2 Design of CNT

We present **CNT**, a system designed for semi-automatic translation of CWL workflows into Nextflow workflows. **CNT** consists of three stages:

- **Tool-level translation (§3.2.1).** This stage constructs the smallest workflow building block by translating a CWL’s CommandLineTool into Nextflow’s Process, specifically by capturing and translating essential fields defined in CWL and then arranging these building blocks into a subworkflow in the next stage.
- **Graph-dependency analysis (§3.2.2).** Following that, **CNT** addresses three important ordering challenges: the order of input/output variables in a Nextflow process, the order of processes in a subworkflow, and the order of subworkflows in the final

| CommandLineTool Field | Template Placeholder     |
|-----------------------|--------------------------|
| ID [ $E$ ]            | Process Name             |
| Requirements [ $E$ ]  | Container                |
| Inputs [ $C$ ]        | Input, Optional Variable |
| Outputs [ $C$ ]       | Output                   |
| baseCommand [ $C$ ]   | Script                   |
| Arguments [ $C$ ]     | Script                   |

Table 3.2: **Summary of value substitution (§3.2.1).** *E*: elementary item that does not require processing, *C*: a collection that requires one before substitution.

translated workflow. CNT solves these challenges simultaneously by doing a recursive exploration technique that captures data lineage information and then builds a directed acyclic graph (DAG), which describes the type signature and order of invocations for both processes and subworkflows.

- **Correctness check (§3.2.3).** The last stage ensures accurate workflow translation by conducting a correctness check for both tool-level translation and graph-dependency analysis. CNT utilizes *typecasting*, a method to confirm the use of proper data types for Nextflow processes and operators.

CNT performs all these stages using only a JSON representation of the original CWL files. The following subsections explain the above three stages in detail.

### 3.2.1 Tool-Level Translation

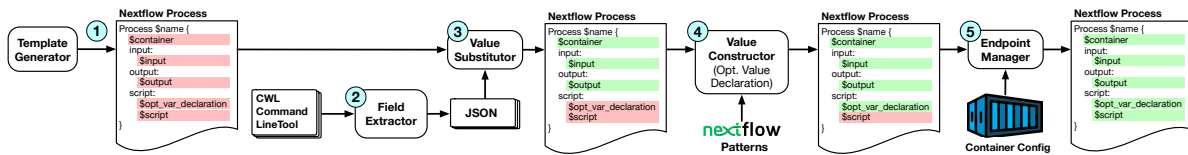


Figure 3.1: **Five steps of tool-level translation (§3.2.1).**

This stage consists of 5 steps: template construction, field extraction, value substitution, optional variable construction, and endpoint management. Fig. 3.1 illustrates the steps

involved in tool-level translation, each of which is denoted by a number from ① – ⑤.

**Template generation.** We observed that Nextflow process structure can accommodate most information from CWL’s CommandLineTool. Moreover, each piece of information (*e.g.*, one input variable) can be separated by a new line – a property that we will exploit in later steps. These features facilitate the generation of a Nextflow Process template (Step ①). The template provides a placeholder for process name, container information, input/output variables, shell commands, and optional variable declarations. Throughout subsequent steps, the template will be filled with corresponding information from CommandLineTool fields, one line for each piece of information.

**Fields extraction.** This step extracts the information needed to fill the placeholders in the template. We identified 6 CommandLineTool fields that are essential for creating a fully functional Nextflow Process: ID, requirements, inputs, outputs, baseCommand, and arguments (Step ②). Information extraction is done by transforming the CommandLineTool source file into a JSON representation, followed by accessing the relevant key-values. CNT uses cwl-utils [42] to transform CWL source files into JSON. Following that, it stores all extracted information in-memory to be processed before writing them to the template placeholders.

**Value substitution.** Table 3.2 summarizes the CommandLineTool fields used for filling each template placeholder along with its type. The value of each field can be in the form of elementary items (*e.g.*, string or integer) or collections (*e.g.*, list or dictionary). For elementary item, CNT directly inserts the value to its respective placeholder (Step ③). For a collection, CNT first flattens any nested items. It then performs additional extractions and constructs a string using the elements from the collection.

**Optional variable declaration.** This step is to support the dataflow model in Nextflow, wherein a Nextflow Process requires explicit configurations of input and can only execute once all its inputs are ready. In contrast, CWL provides for optional inputs, simplifying the writing and execution of workflows (*e.g.*, allowing workflows to run without specifying

every input). There was an effort from the Nextflow community to develop a pattern for addressing CWL’s optional inputs [43]. CNT integrates this pattern to ensure that optional input channels always contain an item, whether or not they are supplied (Step ④).

**Container entrypoint management.** Containerized deployments for scientific workflows have been widely adopted in recent years [40], where each step can be encapsulated within a container (*e.g.*, Docker). This approach requires managing containers’ entry points in order to determine which binary is executed and its corresponding path upon initialization. CNT addresses this issue by examining the configuration file (*e.g.*, Dockerfile) for each container involved in the workflow (Step ⑤), thus ensuring the appropriate binary path is used.

### 3.2.2 *Graph-Dependency Analysis*

The next step in CNT involves composing Nextflow Processes, which were created during tool-level translation, into subworkflows, and combining these subworkflows into a final workflow while addressing ordering challenges. The ordering challenges stem from the difference in how CWL and Nextflow perform invocations: named vs. positional arguments. These challenges can arise in both the Nextflow Process and the subworkflow layer. In the Process layer, the challenge is to manage the order of input/output variables. In the subworkflow layer, the challenge is to manage the order of invocations.

However, to address these challenges, CNT requires information that is scattered across all CWL files, and collecting all the information is not trivial. For example, CNT first needs to collect the path of all related files. Subsequently, it has to extract all necessary information from each file. CNT attempts this through a two-steps approach: recursive exploration for file path collection and DAG construction for the target workflow.

**Recursive exploration file path collection.** CNT first collects the paths of all CWL files used in the workflow. In CWL, a file with class Workflow can include other CWL files

as steps, while a file with `CommandLineTool` can not. The challenge is twofold: 1) CWL allows to use relative path when including another file as a step, and 2) an included file may, in turn, include other files, up to arbitrary depth. To address these challenges, CNT performs a DFS-like recursive exploration technique with the following logic. Each file is considered as the root of a tree with unknown depth. Other files included as steps are the root's children. CNT then examines the JSON representation of each child one by one. If the file is a `CommandLineTool`, CNT extracts its path and then backtracks to its previous operations. However, if the file is a `Workflow`, CNT treats that file as another root and performs the exact same search. This procedure is recursively executed until CNT traverses the whole tree.

**DAG construction for target workflow.** With the collected file paths, CNT extracts all the required information from each file and constructs a DAG for the target workflow. During DAG construction, CNT solves the ordering challenges by ensuring correct order of: 1) I/O variables, and 2) invocations.

1) *Order of I/O variables.* When translating a language that uses named arguments (*e.g.*, CWL) to one that uses positional arguments (*e.g.*, Nextflow), reconstructing type signature requires information from two layers: the callee layer and the caller layer. This is because, with named-arguments, the input/output variables from one layer might appear in different order and aliases in another layer. A translation system can choose either order, but it must ensure consistency across both layers. CNT achieves this by maintaining a DAG data structure. For each callee, CNT creates a *vertex* that contains a map as attribute. The map describes the alias of each of callee's variable as viewed by its caller.

2) *Order of invocations.* After creating vertices, CNT analyzes the relationship between them. A directed edge from a vertex to another is created if an output from the former is used as input by the latter. This completes the DAG construction. Order of invocations is equivalent to topological order of the DAG's vertices.

### 3.2.3 Correctness Check

We observed that when performing translations, CNT needs to take into account Nextflow’s runtime behavior related to the types of input and output variables, which affects Nextflow’s runtime file staging. However, ensuring correct type translation is challenging based on our experience, as there are multiple points in a workflow where a variable may be transformed from a file to another type.

We categorize these possible locations into two layers: Nextflow’s Process layer and workflow layer, each with its own characteristics. In Process layer, undesirable transformations can occur when a variable is passed as either input or output. In the workflow layer, these transformations can take place wherever a Nextflow operator handles the variables.

**Typecasting.** We developed a method called typecasting that ensures correct type translation across both layers.

At the Nextflow Process layer, CNT examines the type declared for each variable within input and output block. It ensures variables intended to represent file objects are correctly declared by inspecting the attributes of DAG vertices associated with the Nextflow Process (§3.2.2). Each Nextflow Process can be associated with at least two vertices: one representing a workflow which invokes it, and the other representing a Nextflow Process itself. As each vertex holds aliases for its respective input and output variables, CNT can determine whether a Process input was originally passed as a file object by examining these aliases. This information is used for declaring variable types.

At the workflow layer, CNT ensures the correct use of Nextflow operators to avoid undesired translations. We also ensure that a variable originating from a file is processed with Nextflow operators that preserve its type, such as using `Channel.path()` instead of `Channel.of()`. To achieve this, CNT considers data lineage by utilizing the DAG constructed in §3.2.2. The origin of a variable can be traced by identifying the edge representing the variable and then traversing back through nodes and directed edges in the DAG.

### 3.3 Manual Translation

While CNT automatically translates a significant portion of CWL codes (detailed in §3.4), some code segments of CWL workflows require manual translation. Manual translation has two main objectives: 1) *ensuring syntax conformance* and 2) *achieving the expected performance*. Syntax conformance aims to reduce errors when converting JavaScript in CWL to Groovy in Nextflow. Achieving the expected performance ensures that the translated workflows deliver the desired performance with proper dataflow and task parallelism.

#### 3.3.1 Ensuring Syntax Conformance

Formal methods [44] for translating JavaScript code into Groovy require constructing a grammar for each language. Since CWL’s adaptation of JavaScript uses a slightly altered grammar (*e.g.*, enclosing inline expressions with ‘\$( )’ or prefixing with ‘|’), for this work we translated JavaScript into Groovy based on a code-level analysis of our CWL workflows. We found that JavaScript expressions within these workflows can be categorized into three common patterns: 1) object attribute access, used to extract values from collection inputs, 2) object method calls, handling simple modifications on literal values, such as string pre/suffix changes and upper/lower case transformations, and 3) ‘actual’ code block, containing tasks ranging from constructing collections and sorting to calculating and managing workflow executions.

Based on these findings, we further enhanced CNT with a three-step approach to assist developers in semi-automatically addressing each pattern. This approach utilizes our carefully designed regex rules to accurately identify each pattern. Subsequently, CNT performs the following three steps.

**Handling object attribute access.** This pattern typically appears as two or three JavaScript tokens separated by dots. As Groovy’s way of accessing object attributes is identical to that of JavaScript’s, CNT can directly apply this expression in the translation.



A challenge for CNT, however, is addressing the many strings appearing before, between, or after successive patterns. Our regex rules effectively tackle this issue.

**Replacing object method call.** The challenge in this step arises from the similarity of this pattern to the previous one. The key difference is that the last token in this pattern represents a method call. To differentiate attributes from method call tokens, we identified frequently used method names, treating them as ‘flags’ for this pattern. For example, in our target workflows, we identified four common method calls: *nameroot*, *tostring*, *basename*, and *dirname*. CNT then maps each method call to its corresponding Groovy equivalent.

**Isolating ‘actual’ code block.** We recognize that the methods applied to the previous patterns are not sufficient for addressing ‘actual’ code blocks due to the extensive variability in JavaScript codes. However, CNT can correctly detect and isolate these code blocks. In its current version, CNT replaces them with a placeholder designed to trigger a syntax error. This placeholder is based on the principle that a clear failure is often better than introducing transient errors. Furthermore, this placeholder acts as a signal for developers, indicating the need for manual translation of the code block into Groovy.

Although resolving this challenge still requires manual intervention by developers, our experience indicates that CNT can significantly reduce the development time by roughly 75%.

### 3.4 Evaluation of CNT

We built CNT in Python, leveraging the `cwl-utils` [42] that facilitates parsing CWL into a JSON representation. We evaluated CNT’s performance across two dimensions: translation MD5 similarity and coverage by employing two widely-used genomic workflows, namely RNA-Seq and DNA-Seq. We then measured the performance gain of adopting CNT by comparing the execution speedup and CPU utilization increase against a popular CWL execution engine.

|     | # Files | MD5 Simil. | O   | OV  | S    |
|-----|---------|------------|-----|-----|------|
| RNA | 31      | 71%        | 0%  | 22% | 100% |
| DNA | 46      | 63%        | 55% | 22% | 44%  |

Table 3.3: **Translation MD5 similarity for automated-translation of CNT (§3.4.1).** Column names "O", "OV", and "S" are shortnames for template placeholders described in §3.2.1. O: Output, OV: Optional Variable, S: Script.

### 3.4.1 Translation MD5 Similarity

CNT successfully performs tool-level translation using the ‘MD5 similarity’ metric, where the **upper bound for perfect similarity is 74%** for RNA-Seq (because JavaScript code with an unhandled pattern will cause an execution error). **CNT is able to achieve 71% similarity** for RNA-Seq.

We define ‘MD5 Similarity’ as having an identical MD5 value between the outputs of CWL and its Nextflow translation. We focused on evaluating the correctness of workflow steps translated by the tool-level translation (§3.2.1), because workflow-level translation often involves manual intervention which leads to different MD5 values. Furthermore, some outputs may contain execution timestamps, resulting in a unique MD5 value for each execution. To address this, we removed the timestamp from these outputs before calculating their MD5 value, or exclude them if removal is not possible (*e.g.*, binary output format).

For any translation errors, we categorize the cause into three groups based on the template placeholder that causes it: output (O), optional variable (OV), and script (S). A single translation mistake can involve multiple placeholders, in which case we increment counts for each affected placeholder. We omitted input and container placeholders from the results since they did not exhibit any translation errors.

**Table 3.3** reports the translation MD5 similarity for two workflows. The ‘# Files’ column represents the number of CommandLineTools files invoked during execution, while the ‘MD5 Simil.’ column indicates the percentage of files producing correct outputs. Template placeholder columns (*e.g.*, O, OV, S) show the percentage of translation errors attributed

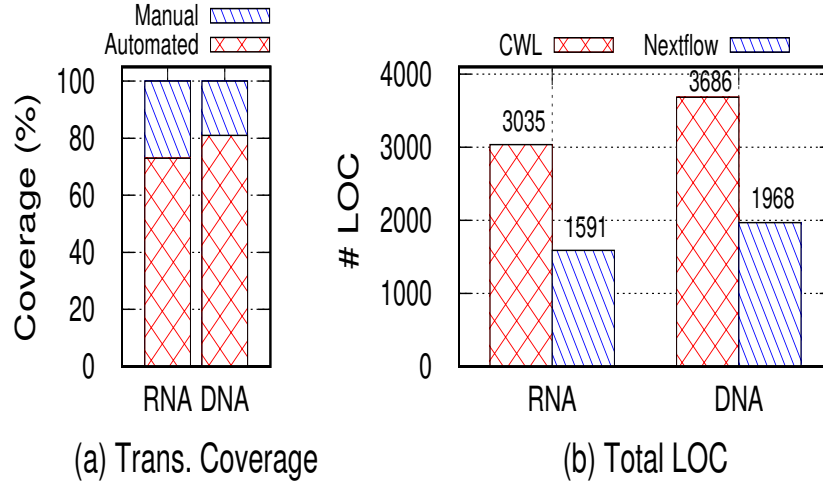


Figure 3.2: **Coverage and LOC evaluation results.** (§3.4.2) (a): *x-axis: workflow types, y-axis: the percentage of coverage.* (b): *x-axis: workflow types, y-axis: the total LOC in thousands.*

to each placeholder. For example, for DNA-Seq workflows, O=55% means 55% of incorrect translations have mistakes in the output placeholder.

The results have several interesting findings. We discuss only one due to the page limit. We observed that no incorrect translations in RNA-Seq workflow occurred on the output (O) placeholder, while the value was around half for DNA-Seq. This indicates DNA-Seq’s way of declaring output is generally more challenging to handle by CNT compared to RNA-Seq. Our in-depth manual inspection revealed that while RNA-Seq’s output declaration heavily utilized JavaScript, the majority aligned with our generalized JavaScript patterns (§3.3.1). On the other hand, the JavaScript patterns in DNA-Seq are more complex, such as `nameroot.split('_').slice(0,4).join('_')`.

### 3.4.2 Translation Coverage and Total Lines of Code (LOC)

**Fig. 3.2a** shows the translation coverage of CNT for both RNA-Seq and DNA-Seq workflows, **achieving 73% and 81% coverage**, respectively. In this figure, the lower portion represents the fraction of automated code translation, while the upper portion denotes manually

written code. The automatically translated part aggregates code from tool-level translation, graph dependencies, typecasting, and partially-handled JavaScript. The manual part includes all fixes and fine-tuning needed to ensure syntax conformance and achieve the expected performance.

The results indicate that CNT had about 8% lower coverage for RNA-Seq than for DNA-Seq, leading to a larger portion of code being manually translated despite RNA-Seq’s more accurate translation correctness. Our further analysis revealed that RNA-Seq has substantial volume of code from CommandLineTool files which requires manual replacements with Nextflow’s operators.

**Fig. 3.2b** compares the total lines of code (LOC) of CWL workflows with their corresponding Nextflow translations, including both automated and manually translated code. We define the total LOC as the cumulative LOC across all files in a workflow. To calculate the LOC of a single file, we first took the total line count and deducted lines comprised solely of comments and whitespaces. Although our approach might count multiple complex JavaScript or Groovy statements on a single line (*e.g.*, separated by ‘;’) as a single LOC, we found that such occurrences are exceptionally rare in practice.

### 3.4.3 Performance Gain by Adopting CNT

Our evaluation primarily focused on two key metrics: speedup (execution time reduction) and CPU utilization improvement when processing the same workflow using cwltool [6] and Nextflow. Cwltool, the open-source official reference for CWL, was chosen because of its wide adoption in the community. Our evaluation shows an **average speedup of 52.5% for RNA-Seq and 30% for DNA-Seq** when migrating from a cwltool-based system to a Nextflow-based system. Additionally, the Nextflow-based system showed an **average increased CPU utilization of 65% for RNA-Seq and 25.5% for DNA-Seq**, in comparison to a cwltool-based system. Detailed range of speedup and utilization

increase can be found in **Table 3.1**.

### 3.5 Conclusion

This paper presents **CNT**. To the best of our knowledge, **CNT** is the first semi-automatic translator that converts CWL workflows into Nextflow workflows. The core of **CNT** is an automatic translation module that performs tool-level translation, graph dependency analysis, and correctness checks to provide highly automated translation coverage, significantly reducing the development time. Additionally, **CNT** has a module to aid manual translation. In particular, **CNT** can accurately identify three common JavaScript patterns in CWL workflows that can further guide developers during translation. Our evaluation of **CNT** centered on its translation accuracy, coverage, and the performance enhancements achieved by using **CNT** to transition from cwltool to Nextflow. Our findings highlight that **CNT** not only ensures precise and extensive translations but also significantly boosts workflow execution speeds and utilization, leading to significant improvement in bioinformatics job processing throughput. In summary, our evaluation demonstrates the value of **CNT** when transitioning from CWL to Nextflow.

# CHAPTER 4

## GENOX: EXPLOITING THREAD SIGNALS FOR DYNAMIC CPU SCALING IN GENOMICS WORKFLOWS

### 4.1 Overview

Genomics workflows are composed of a set of bioinformatics applications through DAG-structured pipelines, processing gigabytes to terabytes of omics data while competing for CPU resources in shared clusters [20, 45, 46]. As large-scale workflows often consume thousands of CPU hours [1, 47], CPU allocation becomes more than merely an optimization problem. Such an allocation decision can have significant cost implications on the large-scale processing of genomic data. For example, suboptimal CPU allocations propagate through the pipeline, increasing computational costs.

However, automatically determining the right CPU quota remains a significant challenge due to the unique characteristics of genomics workflows. For example, genomics workflows often exhibit substantial performance variability that can be hard to predict, depending on input characteristics. As shown in Fig. 4.1a, increasing input size causes execution times for some genomics applications (*e.g.*, STAR [48]) to scale linearly, whereas others (*e.g.*, GATK-br [49]) can experience up to a 10 $\times$  increase in runtime. This increase in workflow execution time often results from dynamic changes in the workflow’s critical path, which is the sequence of operations that dominates total runtime, depending on the input. As shown in Fig. 4.1b and 4.1c, an application that takes  $\leq 10\%$  of runtime on small inputs may consume up to 60% on larger inputs. Unfortunately, these shifts are difficult to predict without executing each specific dataset.

Furthermore, since multiple workflows are executed concurrently in production environments [22, 50], contention-induced slowdowns become increasingly difficult to detect and mitigate. Under contention, traditional CPU utilization metrics often fail to accurately

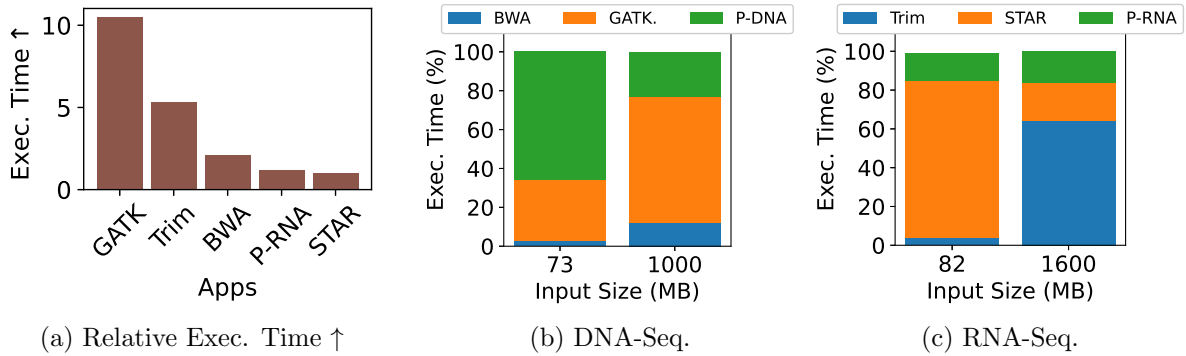


Figure 4.1: **Input-dependent performance variability.** *Genomics applications exhibit varying runtime growth with input size, up to 10 $\times$  differences (4.1a) and shifting critical paths in DNA-Seq (4.1b) and RNA-Seq (4.1c).*

capture true CPU demand. These metrics may report low utilization during I/O stalls or millisecond-scale idle bursts. I/O stalls occur when threads are blocked, waiting for disk or network operations to complete. Millisecond-scale idle bursts occur when applications rapidly alternate between computation and I/O phases at fine time scales, resulting in rapid fluctuations in CPU usage. Therefore, these patterns can make utilization metrics misleading, as they may indicate low CPU usage even when applications have pending computational work. Finally, genomics workflows can include conditional branches, where certain applications run only if the input data meets specific conditions [22, 28]. For example, low-quality genomics data triggers additional refinement steps using data quality control tools like FastQC [51] or Trimmomatic [52], whereas high-quality data bypasses these stages entirely. As a result, resource demands can shift suddenly and unpredictably during execution.

Current CPU allocation approaches, *e.g.*, utilization-based CPU allocators [9–11], ML-based predictors [12–14], or kernel-integrated approaches [15, 16], are often inadequate for genomics workloads. These approaches either react with a delay to dynamic CPU demands [53, 54] or require extensive historical data for predictive model training [55, 56] that *may be infeasible* to obtain (given the high variability of genomic input data) or demand application/kernel-level modifications, which are incompatible with existing bioinformatics

tools.

To understand these challenges quantitatively, we conducted an in-depth characterization study of widely used computational genomics workflows, including DNA sequencing (often abbreviated to DNA-Seq) and RNA sequencing (RNA-Seq) [4]. Our measurements reveal three key characteristics. Specifically, we identify *inflatable parallelism*, where thread counts surge up to  $96\times$ ; measure *stall heterogeneity*, causing up to  $2.1\times$  slowdown from resource contention; and observe *millisecond-scale idle bursts*, which make utilization metrics inherently unreliable for quota decisions under contention.

These findings guide us to establish three key requirements for managing CPU quotas in genomics workflows. A solution must be (i) **responsive to thread bursts**, responding immediately to inflatable parallelism without waiting for (or confirming) utilization changes; (ii) **robust under contention**, maintaining accurate quota decisions despite heterogeneous stalls and idle bursts; and (iii) **adaptive to workflow dynamics**, adjusting to conditional branches and varying application sets without manual intervention.

We present GENOX (GENOmics eXecution), an automated CPU scaling system that addresses these requirements by exploiting thread signals as the primary indicator of compute demand. Our key insight is that thread count can serve as a primary indicator of resource needs in genomics applications, providing immediate, stall-agnostic signals that remain stable even under contention. GENOX is composed of three components: (1) a **thread-based scaler**, which allocates CPU quota proportionally to active threads, detecting CPU demand phase changes immediately as threads are created; (2) a **fine-grained thread monitor**, which operates at 5 ms intervals to capture millisecond-scale dynamics while remaining robust against misleading utilization signals; and (3) a **container-native controller**, which seamlessly incorporates with genomics workflows/applications via standard cgroup interfaces.

We evaluate GENOX in production-like environments using widely adopted genomics



workloads, including both microbenchmarks with representative applications and macro-scale experiments under realistic contention scenarios. Our micro-benchmarks with three representative applications (*e.g.*, BWA [57], STAR, BamSorMaDup [58]) confirm that GENOX effectively addresses all three core requirements (responsiveness, robustness, adaptiveness). GENOX reacts immediately to thread bursts in BWA’s inflatable parallelism phases, maintains stable allocations despite STAR’s millisecond-scale idle bursts, and efficiently adapts to BamSorMaDup’s constant parallel-thread execution, collectively validating our design principles.

Moreover, macro-level evaluation shows that GENOX consistently outperforms state-of-the-art baselines [9–11] under realistic contention, achieving up to 50% reduction in execution time and minimizing straggler slowdowns that reach 200% in baseline approaches. At the workflow level, GENOX improves end-to-end workflow completion time by over 20%. GENOX’s performance gains are stable against increasing input sizes and different input characteristics.

This work has the following research contributions.

1. **Comprehensive characterization of genomics workflow dynamics**, identifying inflatable parallelism, stall heterogeneity, and millisecond-scale idle bursts as fundamental challenges for CPU allocation (§4.2).
2. **Design of a measurement-driven CPU quota allocation system** that uses threads as a stable, leading signal of compute demand, with fine-grained monitoring at 5ms intervals to detect true demand while minimizing resource-demand noise (§4.3.2, §4.3.3).
3. **Non-invasive, production-ready implementation** that integrates containerized workflow engines using only standard cgroup interfaces (§4.3.4).
4. **Comprehensive and thorough evaluation confirming GenoX’s performance**

**improvements** over utilization-based approaches across diverse genomics workloads under realistic contention scenarios (§4.4).

## 4.2 Challenges and Measurement

The unique runtime characteristics of genomics workflows present significant challenges for effective CPU quota allocation. To investigate these challenges, we conducted measurements on two representative genomics workflows: DNA-Seq and RNA-Seq. These workflows integrate widely used genomics applications (*e.g.*, BWA, STAR, GATK, Trimmomatic) that dominate the runtime of production workloads.

We executed the workflows on the Chameleon Cloud [59], using servers with 96 CPU cores and 192 GB memory. Nextflow orchestrated execution with each application running in Docker containers. We monitored runtime metrics through kernel interfaces, collecting per-container CPU quotas, thread counts, and pressure stall information (PSI) [60]. We varied dataset sizes up to 12× to capture input-dependent effects.

Based on these measurements, we reveal three key challenges: (i) **input-dependent performance variability** (§4.2.1), (ii) **contention-induced slowdown** (§4.2.2), and (iii) **dynamic workflow branching** (§4.2.3). For each challenge, we conduct measurements to quantify its impact, reveal unique execution patterns, *e.g.*, *inflatable parallelism* and *stall heterogeneity*, and derive design insights that motivate our solution.

### 4.2.1 Input-dependent Performance Variability (C1)

Resource allocation for workflows should prioritize applications on the critical path to minimize their execution time. However, accurately predicting the completion time of genomics workflows remains challenging due to their complex execution characteristics. In particular, as input size increases, some workflows exhibit linear scaling, while others show up to a 10× increase in runtime beyond what linear scaling would predict. This heterogeneous, non-

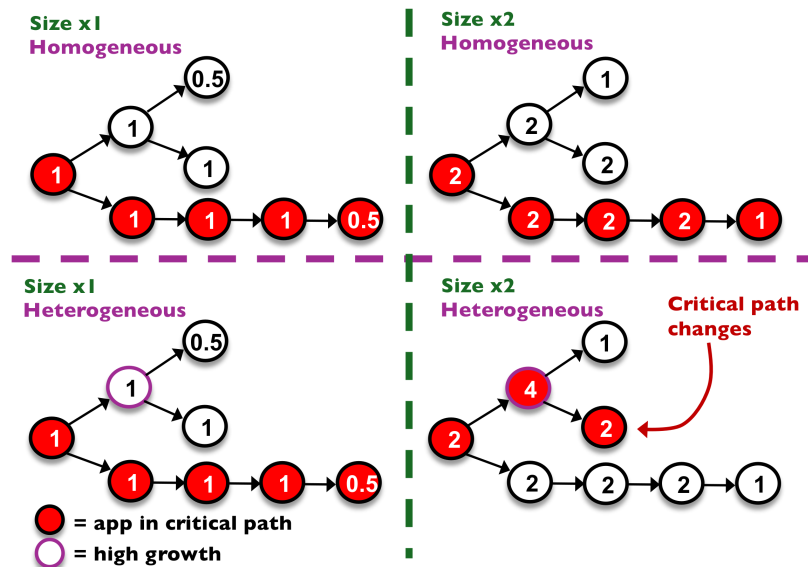


Figure 4.2: **Critical path shifts with input size increase.** ‘Top’: *Homogeneous scaling maintains the same critical path (as input size doubles).* ‘Bottom’: *Heterogeneous scaling causes the critical path to shift between applications (from blue to purple), complicating CPU quota allocation.*

linear scaling shifts the critical path across applications, making it difficult to identify which applications will dominate runtime and complicating decisions for CPU quota allocation.

Fig. 4.2 explains this phenomenon conceptually. The top cases depict *homogeneous growth*, where all applications scale uniformly and the critical path remains unchanged as input size doubles. By contrast, the bottom cases depict *heterogeneous growth*, where differential scaling causes the critical path to shift to a different application. This heterogeneous behavior is what we have observed in production genomics workflows.

To quantify this challenge, we analyzed the 5 longest-running applications in both DNA-Seq and RNA-Seq workflows, measuring execution time growth under identical input size increases. Our results (Fig. 4.1 in §??) reveal that application run times increased by 22% to 232% relative to their baseline, with some experiencing up to 10× greater increase than others (Fig. 4.1a). This uneven scaling shifts the critical path (the application dominating runtime) as input size grows (Fig. 4.1b, 4.1c). In production environments where inputs

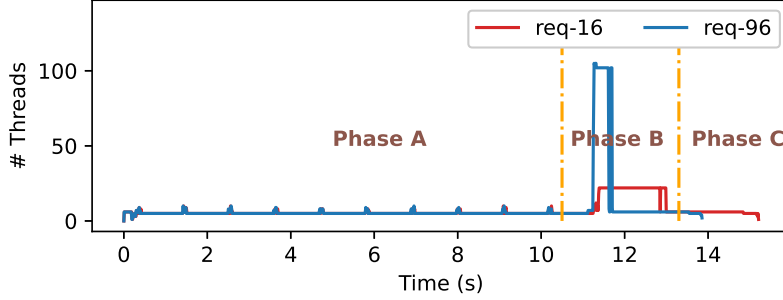


Figure 4.3: **Inflatable parallelism** (‘Phase B’) in BWA execution. *Thread count dynamics across different phases for two configurations (16 and 96 threads). ‘Phase A:’ I/O-driven spikes, ‘Phase B:’ thread counts surge proportionally to user requests, ‘Phase C:’ a stable thread levels until completion. Phase B reveals configuration’s impact on resource consumption and execution time.*

reach hundreds of GB, these shifts become pronounced and further complicate CPU quota allocation.

**Finding #1: Inflatable parallelism.** Our measurements reveal that parallel genomics applications (*e.g.*, BWA) exhibit distinct execution phases corresponding to different computational stages. We consistently observe a phase with a phase with sharp thread count surges, where threads increase dramatically based on user configuration. We introduce the term *inflatable parallelism* to describe this behavior, where thread counts can suddenly expand based on user configuration, creating unpredictable resource demands. Fig. 4.3 illustrates this behavior for BWA, which proceeds through three phases: Phase A: recurrent small spikes, coinciding with I/O activity; Phase B: inflatable parallelism stage with thread surge proportional to the user configuration; Phase C: a drop followed by a stable number of threads until completion.

Only Phase B is sensitive to the user-specified configuration, causing variations in thread counts and overall execution time. This inflatable parallelism poses a significant challenge. *While proportional scaling may seem predictable, its impact on workflow runtime is substantial.* The inflated phase often dominates execution time and can shift the critical path as inputs grow. Existing utilization-based policies fail to timely detect these sudden inflations because utilization lags behind thread creation. By the time high utilization is detected, the

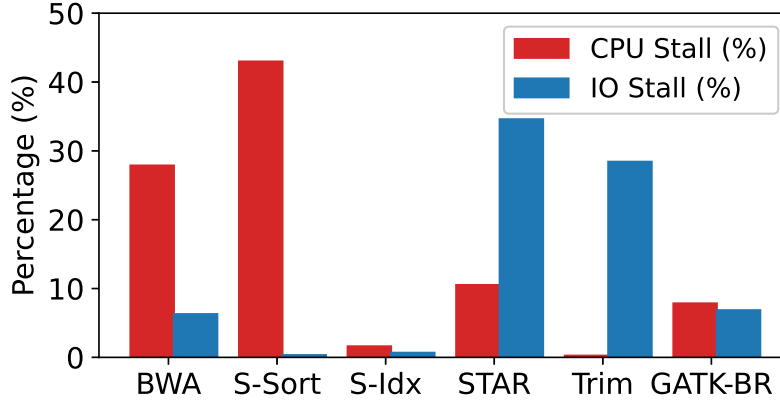


Figure 4.4: **Stall heterogeneity across genomics applications.** *Distribution of CPU and I/O stall percentages showing diverse resource bottlenecks. Some applications are predominantly CPU-bound while others are I/O-bound, complicating utilization-based allocation under contention.*

application has already entered its resource-intensive phase, resulting in allocation delays and performance penalties.

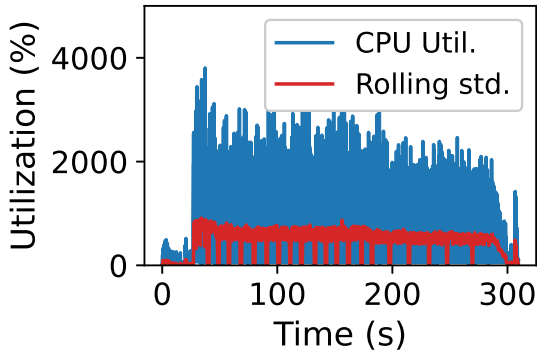
#### 4.2.2 Contention-induced Slowdown (C2)

Genomics workflows are often executed in batches [1], leading to the colocation of multiple applications on the same machine. Such colocated executions inevitably create contention, causing slowdowns as applications interfere with one another’s resource usage. While resource allocators should mitigate this contention, genomics applications exhibit two characteristics that make traditional utilization-based detection ineffective: (i) *stall heterogeneity* and (ii) *millisecond-scale repetitive idle bursts*.

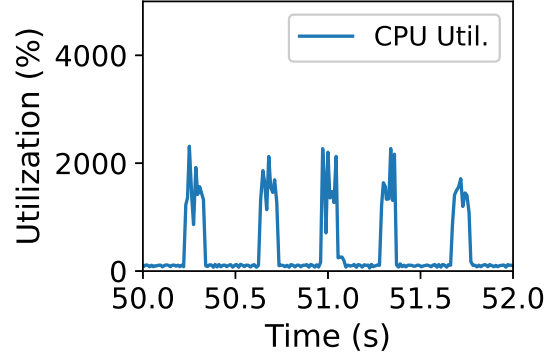
**Stall heterogeneity.** Genomics applications differ widely in their dominant resource usage. Some applications are CPU-bound, others I/O-bound, and many alternate between the two. Fig. 4.4 shows the distribution of CPU and I/O stall time as reported by Linux PSI. Depending on the application, between 10% and 50% of execution time can be lost to stalls on different resources. This heterogeneity makes utilization an unreliable signal. For example, a high utilization reading may indicate genuine compute demand that would benefit from more

Table 4.1: **Contention-induced slowdowns across application pairs.** *Entries show normalized execution time; larger values indicate higher slowdown. Trimm.: Trimmomatic, P-DNA: Picard-DNA, P-RNA: Picard-RNA*

|                | Trimm. | STAR  | P-RNA | BWA   | GATK-br | P-DNA |
|----------------|--------|-------|-------|-------|---------|-------|
| <b>Trimm.</b>  | 1.4×   | —     | —     | —     | —       | —     |
| <b>STAR</b>    | 1.1×   | 2×    | —     | —     | —       | —     |
| <b>P-RNA</b>   | 1.03×  | 1.01× | 1.08× | —     | —       | —     |
| <b>BWA</b>     | 1.07×  | 1.1×  | 1.05× | 2.1×  | —       | —     |
| <b>GATK-br</b> | 1.08×  | 1.05× | 1.05× | 1.13× | 2.02×   | —     |
| <b>P-DNA</b>   | 1.15×  | 1.04× | 1.02× | 1.11× | 1×      | 1×    |



(a) CPU Utilization



(b) Zoomed 2-sec. Window

Figure 4.5: **Millisecond-scale idle burst dynamics in GATK.** (a): CPU utilization shows large volatility over time. (b): fine-grained oscillations from alternating compute and I/O phases (b).

CPUs, or it may simply be inflated by I/O-induced stalls and scheduling delays, in which case allocating additional CPUs would not improve performance. When such applications are colocated, interference leads to non-uniform slowdowns. Table 4.1 shows that co-execution slows down genomics workflows by up to 2.1×

**Millisecond-scale repetitive idle bursts.** Genomics applications frequently interleave compute-intensive phases with short I/O operations, producing millisecond-scale repetitive idle bursts. Fig. 4.5a shows GATK’s CPU utilization fluctuating sharply with 500% – 800% standard deviation. These bursts are not genuine demand shifts but artifacts of how applications alternate between I/O and computation (Fig. 4.5b). Under contention, such

bursts further destabilize utilization as a scaling signal, creating the illusion of rapid demand changes.

**Finding #2: Utilization misleads under contention.** Taken two characteristics (stall heterogeneity and millisecond-scale repetitive idle bursts) together, these observations suggest that utilization is an unreliable indicator when applications are colocated. It cannot distinguish compute demand from I/O stalls, nor differentiate genuine idle periods from brief I/O waits. This finding necessitates a stall-agnostic, phase-aware signal for robust quota allocation.

### 4.2.3 *Dynamic Workflow Branching (C3)*

Finally, resource allocation for genomics workflows faces an additional challenge: dynamic workflow branching. Workflows often contain conditional branches that activate or skip certain applications based on specific input characteristics. These branches arise from conditional logic triggered by input type, input characteristics, input existence, or input metadata. Developers incorporate such logic to selectively perform pre-/post-processing or to tune algorithms based on input characteristics.

Our analysis of widely used harmonization workflows [4] shows that such branching is common, and its runtime impact varies with workflow complexity. In DNA Seq workflows, we observed that a conditional Picard step can add about 5% to execution time. While modest in isolation, the effect compounds in production workflows with multiple conditional steps, making accurate resource prediction increasingly difficult.

**Finding #3: Dynamic thread demand from conditional execution.** The set of active applications can change at runtime based on input characteristics, causing workflows to exhibit shifting resource demands and thread dynamics that cannot be predicted a priori. This unpredictability undermines static allocation strategies and necessitates runtime adaptivity.

#### 4.2.4 Measurement Summary and Design Insights

In this section, we revealed fundamental limitations of utilization-based policies, which motivated us to design a new solution to automatically adjust CPU quotas more effectively. To succeed, such a solution must satisfy the following requirements.

First, it must be **responsive to thread bursts**. Genomics applications exhibit input-dependent variability with sudden thread inflations (inflatable parallelism), requiring phase-aware signals for immediate response. Second, it must be **robust under contention**. When applications are co-located, diverse stalls (CPU vs. I/O) and millisecond-scale repetitive idle bursts make utilization noisy and misleading. A robust system must rely on stall-agnostic signals and operate at fine-grained intervals (*e.g.*, 5 ms) to avoid misallocation. Finally, it must be **adaptive to workflow dynamics**. Workflow execution paths can change at runtime due to conditional branching, activating different applications depending on input size and/or characteristics. An adaptive system continuously monitors per-application threads and adjusts quotas accordingly, eliminating the need for costly offline profiling.

### 4.3 GenoX Design

We design GENOX to address the challenges in §4.2. As shown in Fig. 4.6, GENOX operates as a sidecar alongside the workflow engine, monitoring genomics applications and dynamically adjusting CPU quotas at runtime. It includes three core components: a thread-based scaler (§4.3.2), a fine-grained monitor (§4.3.3), and a container-native controller (§4.3.4).

#### 4.3.1 GENOX Architecture Overview

Fig. 4.6 illustrates how GENOX integrates into end-to-end genomics workflow execution. A workflow begins when a user or bioinformatician (A) prepares application scripts (B) and provides genomics input files (C). These are combined into workflow code (D), which is



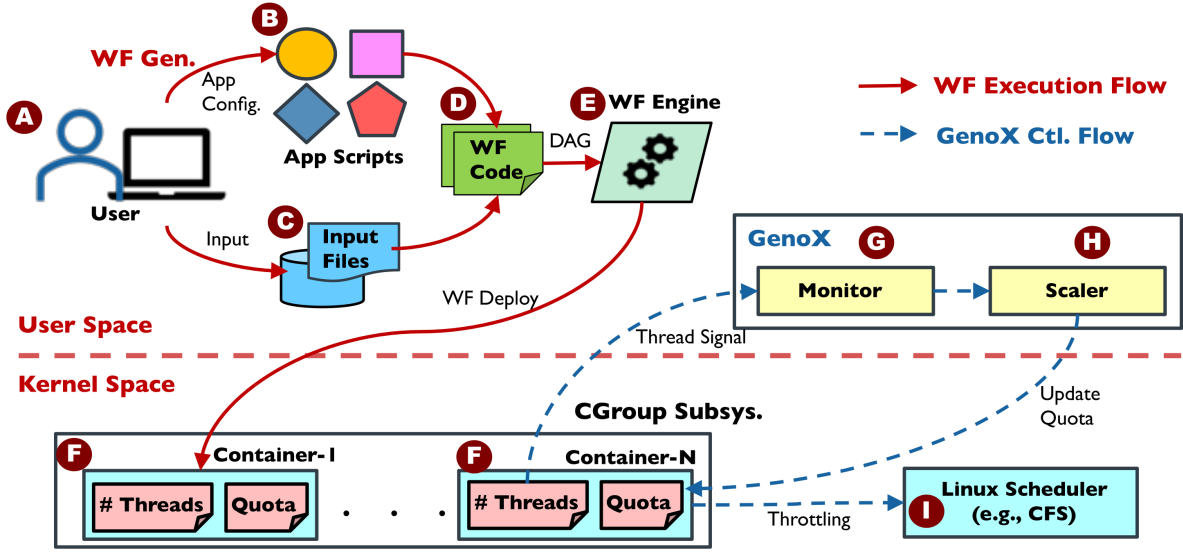


Figure 4.6: **GenoX architecture.** GENOX operates alongside the workflow engine as a sidecar: its monitor collects thread counts, the scaler computes proportional CPU quotas, and quotas are enforced via Linux cgroups and the kernel’s CFS scheduler while applications run inside containers.

executed by the workflow engine (E) to launch applications inside containers (F).

In parallel, GENOX operates as a sidecar component that continuously monitors the execution. Its monitor (G) collects per-container thread signals at fine-grained 5 ms intervals. The scaler (H) then computes proportional CPU quotas – approximately one core per thread – and applies them via Linux cgroup interfaces. Finally, the kernel’s scheduler (I) enforces these quotas transparently, without requiring modifications to the workflow engine or applications. The next sections describe each of GENOX’s components.

### 4.3.2 Thread-based Scaler for Responsiveness

Choosing the right scaling signal is crucial to GENOX design, as the signal determines when and how much CPU allocation should change to match compute demand. The signal must accurately reflect true demand so GENOX can allocate resources proportionally without performance loss.

**Thread count as a scaling signal.** GENOX uses thread count as its primary scaling signal,

providing immediate response to phase changes. Unlike utilization-based policies that lag behind actual demand, thread count changes immediately when an application enters a new phase. This enables GENOX to detect *inflatable parallelism* (§4.2.1), where applications spawn additional threads in proportion to user-specified settings. By tracking thread counts directly, GENOX responds to bursts promptly at fine granularity. Fig. 4.7 shows how thread counts clearly delineate execution phases in representative genomics applications (*e.g.*, BWA, STAR, and GATK).

**Mechanism.** To implement this, GENOX obtains two kernel-level metrics for each container: the number of active threads and the assigned CPU quota, both accessible through standard cgroup interfaces (*e.g.*, ‘/sys/fs/cgroup’) with negligible overhead. Using this information, the scaler applies an empirically validated proportional heuristic that allocates approximately one CPU core per active thread. This approach balances responsiveness and control overhead while aligning with the Linux scheduler’s granularity. For example, if ten threads are active, the application receives a quota for ten cores; if only two threads remain, the quota is reduced accordingly.

The monitor and scaler operate as a producer-consumer loop forming GENOX’s core feedback mechanism. The monitor (producer) samples per-container thread counts at a short intervals, while the scaler (consumer) recomputes quotas accordingly. This design ensures timely adjustments with minimal overhead, as thread counts are accessed efficiently via cgroup interfaces. The same feedback pattern extends to contention handling in the fine-grained monitor (§4.3.3).

**Impact.** By leveraging thread-level signals, GENOX quickly responds to thread bursts. Unlike utilization-based policies that provide only delayed feedback, GENOX captures phase shifts immediately and reallocates CPU quota promptly at fine granularity. This capability allows GENOX to handle inflatable parallelism effectively, ensuring that critical-path applications receive sufficient CPU resources to make forward progress without bottlenecks.

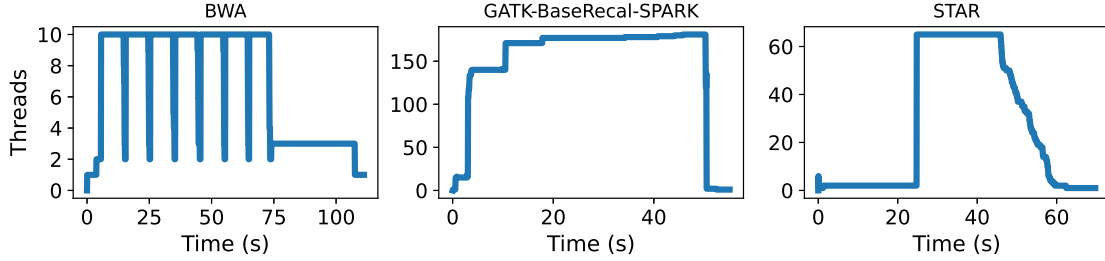


Figure 4.7: **Thread-level dynamics in genomics applications.** *In BWA, STAR, and GATK, thread activity shifts sharply across phases, exposing bursts of parallelism (inflatable parallelism) that utilization signals fail to capture.*

### 4.3.3 Fine-grained Thread Monitor for Robustness

When multiple genomics jobs execute concurrently, contention for shared resources leads to performance variability. Some phases are compute-bound while others are dominated by I/O stalls, creating heterogeneous stall patterns and millisecond-scale idle bursts (§4.2.2). For example, short idle bursts can make utilization appear low even though applications continue to demand CPU time. Utilization-based policies that rely on this unstable signal risk under-allocating quota during critical phases. To ensure robustness, GENOX monitors thread counts as a stable signal that is unaffected by whether stalls originate from computation or I/O.

**Mechanism.** This fine-grained monitor reuses the core feedback loop from §4.3.2, but specializes it for high-resolution sampling under contention. In particular, it records per-container thread counts at a 5 ms interval, which is short enough to capture rapid idle bursts while remaining lightweight. Because active threads remain visible even when blocked on I/O, thread counts provide a stall-agnostic and timely reflection of demand. These measurements are continuously streamed to the scaler for quota adjustment. Fig. 4.8 illustrates this behavior: while coarse-grained monitoring aggregates idle bursts with compute phases making utilization appear moderate, fine-grained monitoring at 5 ms intervals captures the true demand pattern. Thread counts remain stable even as utilization oscillates, enabling GENOX to base decisions on reliable signals.

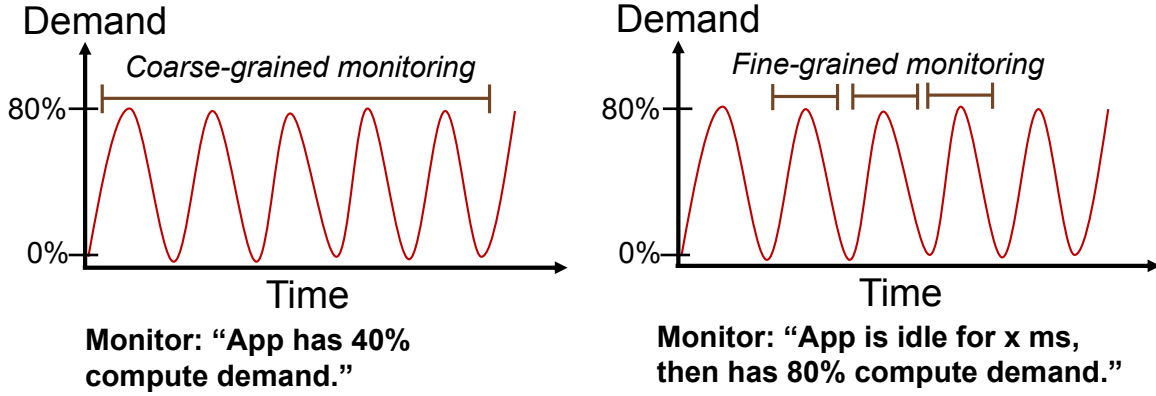


Figure 4.8: **Coarse-grained vs fine-grained monitoring.** *Utilization fluctuates with millisecond-scale idle bursts, while thread counts remain stable, enabling GENOX to base quota decisions on reliable signals.*

**Impact.** By grounding quota decisions in stable thread-level signals, GENOX achieves robustness under contention. It avoids misinterpreting idle bursts as reduced demand and sustains fair allocation across co-located genomics applications, thereby mitigating contention-induced slowdowns.

#### 4.3.4 Container-native Controller for Adaptiveness

Genomics workflows frequently undergo dynamic branching, where execution paths depend on input data or intermediate results (§4.2.3). As a result, new containers may be created on-the-fly, while others may terminate earlier than expected. A static quota plan cannot anticipate such changes, leading to wasted resources or starvation of newly tasks. To remain adaptive, GENOX must handle workflow dynamics without manual intervention or workflow engine modifications.

**Mechanism.** GENOX achieves this adaptiveness through a container-native controller that integrates directly with Linux cgroups. When a workflow engine launches new tasks, they automatically appear as containers within the cgroup hierarchy. The controller detects these

containers in real time and applies GENOX’s quota policies without requiring any workflow-level awareness. Similarly, when containers terminate, their quotas are immediately return to the pool.

The controller operates as a lightweight daemon that monitors the cgroup subsystem for container events. Upon detecting a new container, it attaches quota enforcement hooks so that scaling decisions from the monitor and scaler (§4.3.2, §4.3.3) are applied transparently. Because it leverages existing OS primitives, the controller incurs negligible overhead and requires no modifications to workflow engines or applications.

**Impact.** By coupling quota enforcement to containers, GENOX achieves adaptiveness to workflow dynamics. It automatically incorporates new tasks, releases resources from completed ones, and sustains efficient utilization throughout branching workflows. This design enables GENOX to provide adaptive CPU allocation without requiring changes to genomics applications or workflow engines.

## 4.4 Evaluation of GenoX

We evaluate GENOX in production-like settings using production-scale datasets and complete DNA-Seq and RNA-Seq pipelines. Our evaluation assesses three criteria: (i) **responsiveness** to bursty, phase-shifting demand, (ii) **robustness** under contention and I/O stalls, and (iii) **adaptiveness** to workflow dynamics.

GENOX is compared against three representative baselines, capturing widely used heuristics for dynamic CPU allocation. Evaluations cover both micro-benchmarks (case studies) and macro-scale results. The case studies (§4.4.2) validate GENOX’s mechanisms on representative applications, while macro-scale experiments (§4.4.3, §4.4.4) quantify application-level and workflow-level improvements under realistic contention. We also evaluate GENOX’s performance projection on increasing input sizes and performance sensitivity against inputs with different characteristics. We report execution time, straggler slowdown, and end-to-end

workflow runtime.

#### 4.4.1 Evaluation Methodology

**HW setup.** All experiments are conducted on the Chameleon NSF Cloud testbed, using a server equipped with 96 CPU cores, 200 GB of memory, and 500 GB of SSD storage. Workflows are containerized and orchestrated via Nextflow.

**Baselines.** We evaluate GENOX against three baselines widely studied in dynamic CPU allocation.

*Autopilot* [9] monitors recent CPU utilization based on its moving-window averaging strategy and adjusts quotas based on smoothed historical demand. Autopilot’s approach reduces short-term fluctuations and avoids overreacting to momentary noise, but it also delays responses to sudden bursts or phase shifts.

*AutoThrottle* [10] employs reactive scaling to adjust CPU quotas aggressively once utilization crosses predefined thresholds. AutoThrottle responds quickly to bursts but is prone to overshoot and instability when demand fluctuates.

*SHOWAR* [11] uses statistical thresholding (*e.g.*, three-sigma rules) to identify deviations from average utilization. SHOWAR aims to be robust against noise but often underestimates demand in bursty workloads, leading to slowdowns due to stragglers.

**Evaluation metrics.** We use metrics at both application and workflow levels. For application-level studies, we measure per-application execution time and straggler slowdown. To capture straggler effects accurately, we evaluate the completion time of the last-finishing workflow, reflecting the cumulative impact of contention across constituent applications. For workflow-level studies, we measure end-to-end runtime and speedup of complete DNA-Seq and RNA-Seq pipelines, showing how per-stage allocation impacts overall throughput in production workflows. We measure application execution time for studies related to input sizes and different input characteristics.

Table 4.2: **CPU usage patterns of three representative genomics applications.** *Each application is selected to demonstrate a particular design choice of GENOX. i.e., responsiveness (BWA), responsiveness and robustness (STAR), and adaptivity (BamSorMaDup).*

| App          | Exec. Pattern (Thread/CPU)                                       | Eval. Focus              |
|--------------|------------------------------------------------------------------|--------------------------|
| BWA          | Bursty inflatable parallelism with sudden thread inflation       | Responsive.              |
| STAR         | Multi-phase execution with rising /falling threads and idle gaps | Responsive.<br>+ Robust. |
| BamSor-MaDup | Constant parallel execution with fixed 16-thread demand          | Adaptive.                |

#### 4.4.2 Micro-benchmarks (Case Studies)

To validate GENOX’s mechanisms under controlled execution patterns, we conduct microbenchmarks on three representative genomics applications: BWA, STAR, and BamSorMaDup. Table 4.2 summarizes how each application corresponds to the challenges (in §4.2) and evaluates a specific GENOX’s capability: responsiveness, robustness, and adaptiveness.

**BWA (Responsiveness).** We use BWA, a DNA aligner [57], to stress GENOX’s ability to handle *bursty inflatable parallelism* (§4.2.1). The key question is whether GENOX can adjust quotas fast enough to prevent throttling during bursts.

Fig. 4.9a shows CPU utilization under different policies. Utilization-based approaches (Autopilot, SHOWAR, AutoThrottle) lag behind bursts, reacting only after resource consumption changes. GENOX prevents throttling by tracking threads directly. Fig. 4.9b reports CPU quota allocation over time. GENOX increases CPU quota promptly in proportion to thread inflation and scales down once bursts decline. In contrast, utilization-based baselines exhibit delayed reactions. For example, Autopilot, SHOWAR, and AutoThrottle increase quotas only after demand has shifted, resulting in temporary under-allocation. As a result, GENOX minimizes throttling penalties and achieves the fastest completion time among all policies (Fig. 4.9c).

**STAR (Responsiveness + Robustness).** We use STAR, an RNA-seq aligner [48] to test GENOX under *multi-phase execution with idle bursts and heterogeneous stalls* (§4.2.2). The

key challenge is whether GENOX can remain responsive to bursts while being robust against misleading I/O stall signals.

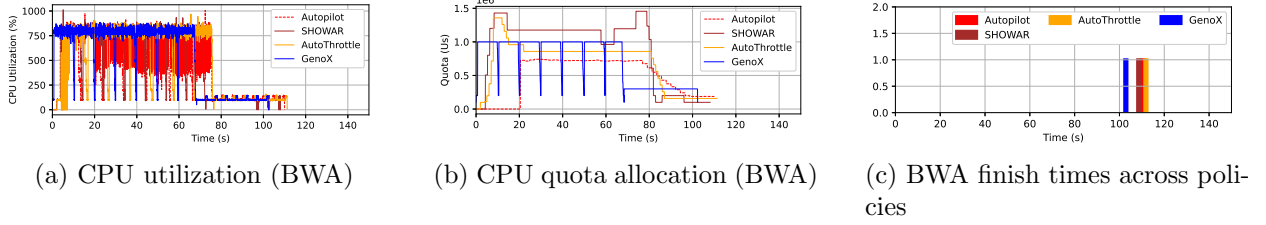


Figure 4.9: **BWA case study results.** ‘(a) CPU utilization:’ Baselines lag behind bursts while GENOX immediately detects thread inflation. ‘(b) Quota allocation over time:’ GENOX scales up/down promptly with thread count whereas baselines react after demand shifts. ‘(c) Completion times:’ GENOX achieves the shortest runtime by eliminating/minimizing throttling penalties, while baselines finish later due to delayed reactions.

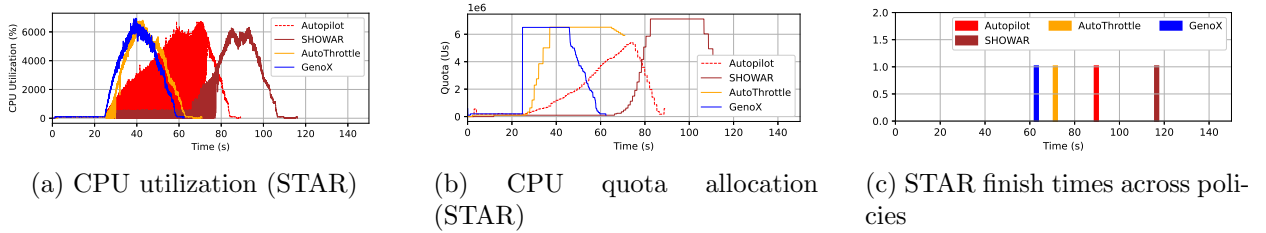


Figure 4.10: **STAR case study results.** (a) ‘CPU utilization:’ Baselines misinterpret idle bursts as reduced demand while GENOX tracks thread counts to maintain accurate allocation. (b) ‘Quota allocation over time:’ GENOX responds immediately to thread changes, whereas baselines exhibit delayed reactions. (c) ‘Completion times:’ GENOX achieves fastest runtime through prompt response and robustness to idle bursts.

Fig. 4.10a shows CPU utilization across policies. Baseline approaches overestimate demand during idle bursts and underestimate it when demand rises quickly. However, GENOX, relying on thread counts rather than utilization, avoids these misinterpretations and maintains a closer alignment with actual demand. Fig. 4.10b reports quota allocation. GENOX scales quotas up immediately as thread counts rise and reduces them once phases complete. In contrast, baselines lag behind demand by adjusting CPU quota only after consumption has shifted. As a result, GENOX achieves the fastest completion time (Fig. 4.10c). By



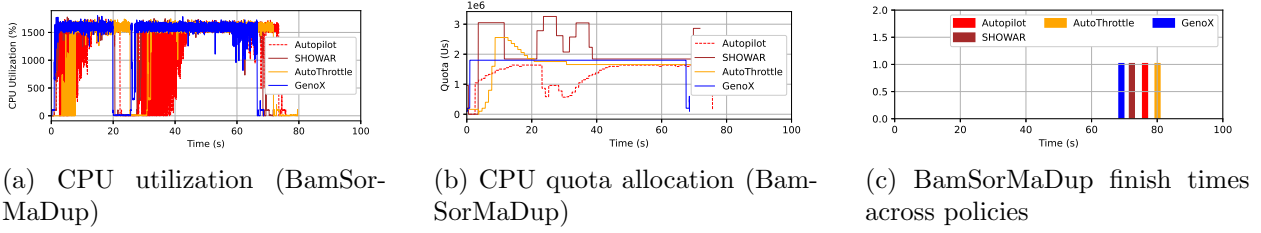


Figure 4.11: **BamSorMaDup case study results.** ‘(a) CPU utilization:’ *UBaselines fluctuate due to I/O stalls while GENOX maintains stability matching the constant 16-thread profile.* ‘(b) Quota allocation over time:’ *GENOX consistently assigns one core per thread whereas baselines make unstable adjustments.* ‘(c) Completion times:’ *All policies finish similarly but GENOX avoids resource waste, demonstrating efficient adaptation to steady workloads.*

preventing over-throttling during bursts and avoiding over-provisioning during idle gaps, GENOX reduces wasted resources and improves efficiency relative to all baselines.

**BamSorMaDup (Adaptiveness).** We use BamSorMaDup, a duplicate-marking tool [58], to evaluate GENOX’s *adaptiveness under steady parallel workloads*, where the thread count remains constant at 16 throughout execution (§4.2.3). The challenge here is whether GENOX can avoid wasting resources in the absence of bursts or dynamic branching.

Fig. 4.11a shows CPU utilization. Baseline policies occasionally misinterpret I/O stalls, leading to oscillations in quota allocation. GENOX, by contrast, detects the constant 16-thread demand and keeps allocations minimal and stable. Fig. 4.11b reports CPU quota allocation. GENOX consistently assigns one core per thread (16 cores total), matching the constant-thread profile. However, the baselines make unstable quota adjustments due to noisy utilization signals, leading to inefficient resource allocation. As shown in Fig. 4.11c, all policies finish within a similar time frame since BamSorMaDup is not CPU-intensive. However, GENOX achieves this without wasting CPU cycles, demonstrating its efficiency and adaptiveness.

**Takeaway.** The case studies show that GENOX addresses three primary challenges: immediate response to thread bursts (case BWA); maintaining stability in the presence of

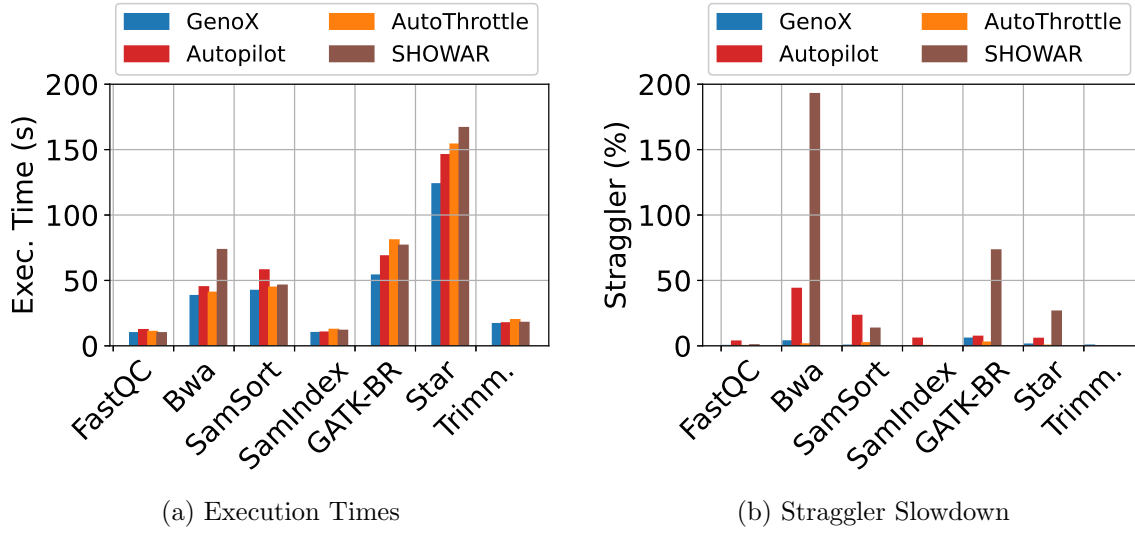


Figure 4.12: **Execution times (4.12a) and straggler slowdown (4.12b) of DNA-Seq under contention.** ‘(a) Execution time:’ GENOX consistently reduces completion times across all applications, whereas baselines show inconsistent performance. ‘(b) Straggler slowdown:’ GENOX prevents performance degradation from stragglers, whereas baselines suffer severe delays (e.g., 200% slowdown for BWA under SHOWAR and 43% under Autopilot.)

I/O-induced noise (case STAR); and efficient adaptation to constant-thread workloads (case BamSorMaDup).

#### 4.4.3 Application-level Performance

We next evaluate GENOX at the application level to assess its performance under resource contention. To introduce contention, we execute multiple concurrent workflows, where each workflow runs several genomics applications that compete for shared computing resources. This experimental setup reflects production environments, where co-running tasks are prevalent and straggler effects can significantly impact overall workflow performance. We measure performance using the completion time of the last-finishing workflow, which captures the impact of contention across all constituent applications.

Fig. 4.12a shows the execution times of DNA-Seq applications under contention. GENOX consistently achieves the shortest completion times and outperforms all baseline methods

Table 4.3: **End-to-end execution times (sec.) for DNA-Seq and RNA-Seq workflows.** GENOX consistently achieves the shortest runtimes, speeding up DNA-Seq by 12 – 18% and RNA-Seq by 19 – 25% compared to baselines.

|                | Autopilot | AutoThrottle | SHOWAR | GenoX |
|----------------|-----------|--------------|--------|-------|
| <b>DNA-Seq</b> | 1085      | 1115         | 1059   | 944   |
| <b>RNA-Seq</b> | 339       | 352          | 335    | 282   |

by effectively adapting to varying CPU usage patterns across applications. In contrast, baseline approaches show inconsistent performance across applications. For example, while SHOWAR achieves the second-best performance for Trimmomatic, it performs worst for BWA. Similarly, Autopilot ranks second-best for Samtools-Index but exhibits the poorest performance for Samtools-Sort. Across all evaluated applications, GENOX achieves the fastest completion times, reducing execution time by up to 49% relative to baseline methods, with an average improvement of 15.7%.

We also measured the impact of stragglers. Fig. 4.12b presents straggler slowdowns, defined as the additional wait time imposed on applications in the slowest workflow relative to those in the fastest (*e.g.*, if the fastest finishes at  $t = 8$  and the slowest at  $t = 10$ , the slowdown is  $(10-8)/8 = 25\%$ ). GENOX effectively avoids such slowdowns, while utilization-based baselines suffer from significant delays. SHOWAR, for example, struggles with BWA, GATK-BR, and STAR, exhibiting nearly 200% slowdown on BWA, where the faster workflow waits almost its entire runtime. Autopilot performs better but still incurs notable slowdowns, including 43% on BWA and 23% on Samtools-Sort. In contrast, GENOX prevents these effects by adjusting quotas promptly, avoiding the accumulation of straggler delays.

**Takeaway.** At the application level, GENOX consistently outperforms baselines and effectively handles contention by: (1) maintaining consistent performance across diverse applications, (2) minimizing straggler effects through prompt quota adjustments, and (3) achieving significant end-to-end workflow speedups, validating GENOX’s thread-based design.

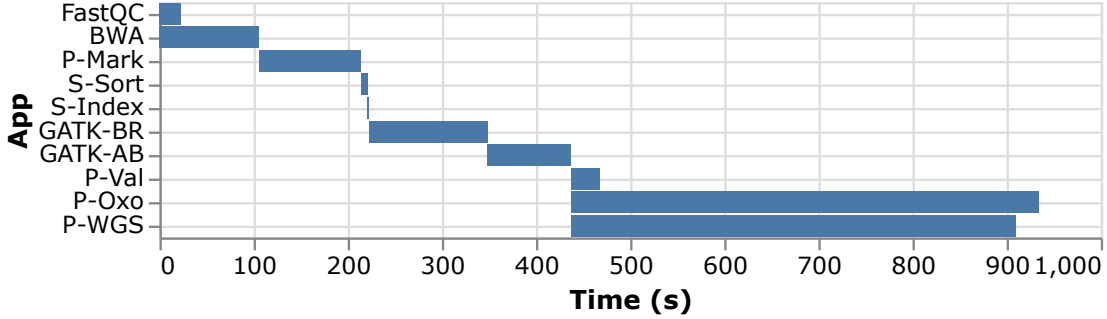


Figure 4.13: **Per-application start and finish times in DNA-Seq.** *Baseline policies prolong key stages (e.g., P-Oxo (Picard-CollectOxoGMetrics) by up to 26%), delaying downstream tasks and inflating overall runtime, while GENOX keeps stage completion times shorter and more balanced.*

#### 4.4.4 Workflow-level Performance

We execute complete DNA-Seq and RNA-Seq pipelines with production-scale datasets to measure GENOX’s workflow-level performance. This evaluation is to capture the cumulative effects of CPU quota allocation decisions across multiple workflow stages. In genomics pipelines, delays in a single application can propagate through dependent tasks, leading to significant increases in overall workflow runtime.

Table 4.3 reports total workflow execution times. GENOX consistently achieves the shortest runtimes by reducing completion time by 12% – 18% for DNA-Seq and 19% – 25% for RNA-Seq compared to baselines. At the workflow level, GENOX was able to maintain the improvements observed at the application level, with average speedups comparable to per-application benefits. Moreover, GENOX was the only approach that kept performance close to unconstrained execution while avoiding the inefficiencies of wasted resources.

Fig. 4.13 and 4.14 report the start and finish times for individual applications within both workflows. Baselines often prolong long-running stages, delaying dependent tasks and inflating overall runtime. For example, STAR in RNA-Seq extends by 19% under SHOWAR, while P-Oxo (PicardCollectQxqGMetrics) [61] in DNA-Seq is prolonged by 26%. These stage-level delays compound throughout the workflow, explaining the performance gaps in

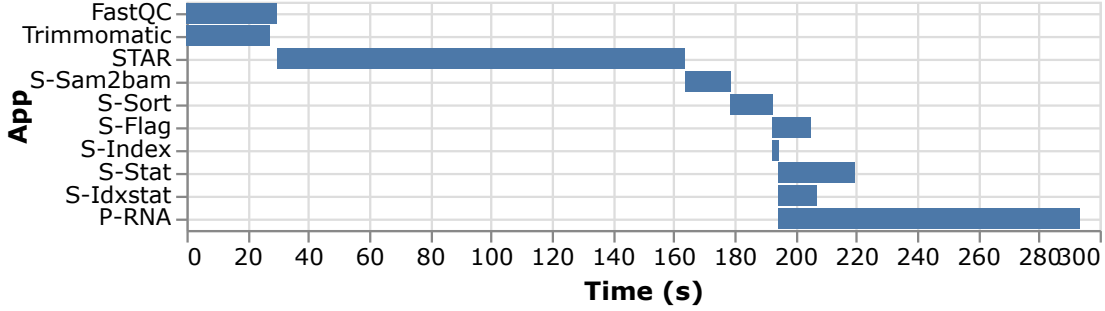


Figure 4.14: **Per-application start and finish times in RNA-Seq.** *Baselines extend long-running stages such as STAR (by 19% under SHOWAR), which cascades into delays for dependent tasks. GENOX reduces such stage-level bottlenecks, sustaining faster progression at the workflow level.*

Table 4.3.

**Takeaway.** At the workflow level, GENOX manages prompt quota adjustments throughout the entire pipeline. By doing so, GENOX avoids the cascading delays that occur when one stage’s slowdown impacts subsequent stages, delivering practical benefits for large-scale genomics processing.

#### 4.4.5 Performance Projection on Increasing Input Sizes

Next, we evaluate the performance of GENOX under increasing input sizes. This evaluation aims to capture GENOX ’s performance projection when input sizes are similar to those seen in production. Here, we synthetically generate input files using a next-generation sequencing data simulator [62] with sizes of 1GB, 16GB, 32GB, 64GB, and 128GBs. Based on our discussion with domain experts, the highest input size we use, namely the 128GB, is >90th percentile of input sizes in production system.

We then evaluate GENOX ’s performance on both DNA-Seq and RNA-Seq, represented by BWA and STAR, using the generated input files. Figures 4.15a and 4.15b show the resulting execution time of each input size on DNA-Seq and RNA-Seq, respectively. On DNA-Seq, we observe that execution time scales linearly with input size on all policies except Autopilot,

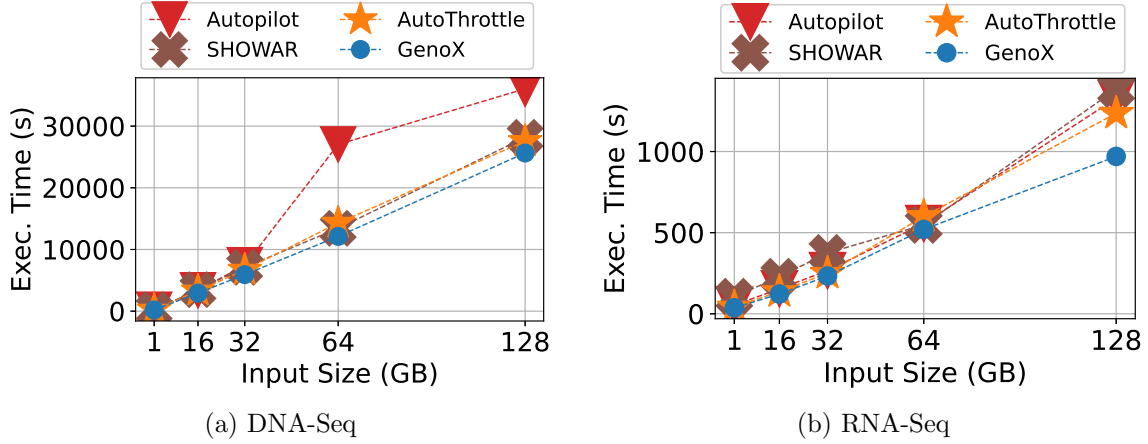


Figure 4.15: **Execution time against increasing input size on DNA-Seq (4.15a) and RNA-Seq (4.15b)** ‘(a) DNA-Seq’ (BWA): GENOX maintains an average of 10-26% speedups on each input size compared to other policies, with an overall average speedup of 16.3%. ‘(b) RNA-Seq’ (STAR): GENOX achieves an average of 8-35% speedups on each input size, with an overall average speedup of 23%.

which suffers from 29-55% slowdown compared to GENOX on high input sizes (64GB and 128GB). Overall, across all input sizes, GENOX is the fastest – it maintains an average of 10-26% speedups on each input size compared to other policies, with an overall average speedup of 16.3%. This advantage can be attributed to GENOX’s responsiveness, robustness, and adaptiveness, while other policies suffer from severe CPU throttling penalties due to delayed reactions in allocation.

Similarly, on RNA-Seq, GENOX maintains the fastest execution time across all input sizes. Overall, GENOX achieves an average of 8-35% speedups on each input size, with an overall average speedup of 23%. Moreover, GENOX’s performance gain over other policies becomes more pronounced as input size increases. We observe that all other policies experience a spike in execution time increase on input size = 128GB. This happens because policies may accumulate more CPU throttling penalties during longer executions. In contrast, GENOX’s efficient allocation minimizes CPU throttling penalties even during long executions. Thus, it maintains a more stable execution time increase, resulting in a pronounced performance gain at input size = 128GB.

Table 4.4: **Details of DNA and RNA datasets.** *The collected datasets have different input characteristics, as represented by diverse experimental strategy, sequencing hardware, GC content, and total file size.*

| Dataset     | Type | Exp. Strategy | Hardware     | GC (%) | Size (GB) |
|-------------|------|---------------|--------------|--------|-----------|
| SRR15522583 | DNA  | WXS           | HiSeq 2000   | 52.9   | 16        |
| SRR15522585 | DNA  | WXS           | HiSeq 2000   | 52.5   | 17        |
| SRR36660861 | DNA  | Chip-Seq      | NextSeq 500  | 39.8   | 6.5       |
| SRR36660862 | DNA  | Chip-Seq      | NextSeq 500  | 40.3   | 7.6       |
| SRR24039105 | DNA  | WGS           | NovaSeq 6000 | 50     | 38        |
| SRR24039107 | DNA  | WGS           | NovaSeq 6000 | 48.9   | 32        |
| SRR38453834 | RNA  | RNA-Seq       | NovaSeq X    | 50.7   | 64        |
| SRR39288312 | RNA  | RNA-Seq       | HiSeq 2000   | 51     | 8.9       |
| SRR6490019  | RNA  | RNA-Seq       | NextSeq 500  | 48     | 16.6      |
| SRR6490047  | RNA  | RNA-Seq       | HiSeq 2500   | 44     | 9.2       |
| SRR39355689 | RNA  | RIP-Seq       | NovaSeq 6000 | 47.1   | 32        |
| SRR39355691 | RNA  | RIP-Seq       | NovaSeq 6000 | 49.7   | 36        |

**Takeaway.** GENOX minimizes CPU throttling penalties even during long executions (i.e. processing large input sizes). As a result, GENOX maintains performance benefits as input sizes increase.

#### 4.4.6 Performance on Varying Datasets

Finally, we evaluate the performance of GENOX using various public DNA and RNA datasets. The evaluation aims to assess GENOX ’s performance sensitivity against inputs with different characteristics. To achieve this, we first collect a total of 12 public datasets from the Sequence Read Archive (SRA) [63], consisting of 6 DNA and RNA datasets. For both DNA and RNA, we choose datasets with diverse experimental strategies, hardware used for sequencing, and GC content as listed in SRA. Table 4.4 describes the details of these datasets. Finally, we process the DNA datasets using BWA and RNA datasets using STAR.

Figure 4.16a and 4.16b shows the execution time of each policy on DNA datasets and RNA datasets, respectively. Overall, we observe that GENOX is able to maintain comparable performance gains similar to previous evaluations on both DNA and RNA datasets, despite

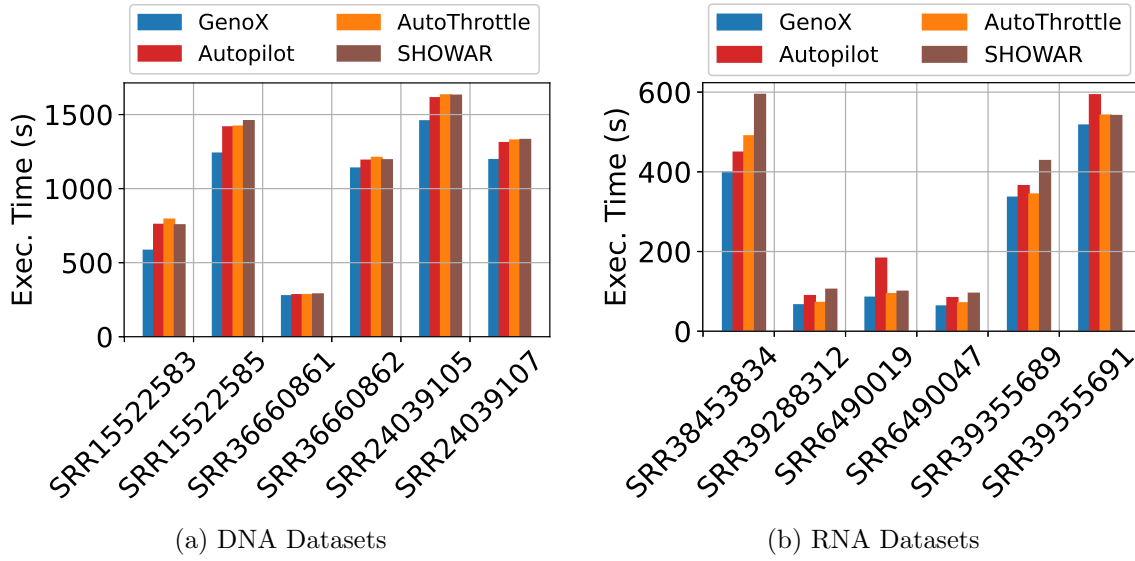


Figure 4.16: **Execution time on various public DNA (4.16a) and RNA (4.16b) datasets.** ‘(a) DNA Datasets:’ GENOX achieves an average of 3.2-24.1% speedups per dataset, with an overall average of 11% across all DNA datasets. ‘(b) RNA Datasets:’ GENOX achieves an average of 8.3-35.6% speedups per dataset, with an overall average of 23% across all RNA datasets.

the existence of different input characteristics. On DNA datasets, GENOX achieves an average of 3.2-24.1% speedups per dataset compared to other policies, with an overall average of 11% speedup across all DNA datasets. The lowest average speedups happen on datasets with Chip-Seq experimental strategy, namely SRR36660861 and SRR36660861, with 3.15% and 5.03% average speedup, respectively. These low speedups might indicate that when processing Chip-Seq datasets, BWA’s CPU demand are relatively more stable and there are less needs for responsive CPU reallocations. As a result, all policies experience minimal CPU throttling penalties, and overall performance become similar.

GENOX’s speedups on RNA datasets are noticeably higher than DNA datasets. On RNA datasets, GENOX achieves an average of 8.31-35.6% speedups per dataset, with an overall average of 23% speedup across all datasets. The lowest average speedups for RNA happen on datasets with RIP-Seq experimental strategy, namely SRR39355689 and SRR39355691, with 10.6% and 7.3% average speedup, respectively.



Table 4.5: **Execution times of DNA-Seq and RNA-Seq workflows without contention.** *In resource-rich environments, GENOX performs nearly identically to Linux’s NoLimit policy, with only about 1% difference.*

|         | NoLimit | GenoX | Differences             |
|---------|---------|-------|-------------------------|
| DNA-Seq | 935     | 944   | 0.5% ↑; GENOX is slower |
| RNA-Seq | 286     | 282   | 1.4% ↓; GENOX is faster |

**Takeaway.** GENOX’s performance gains are not sensitive to different input characteristics. GENOX is able to maintain performance gains similar to previous evaluations despite diversity in inputs’ experimental strategy, sequencing hardware, and GC content.

## 4.5 Discussion

We discuss GENOX’s effective scenarios and limitations.

**Effective Scenarios for GenoX.** Our results suggest that GENOX is most effective in dynamic and contention-heavy workflows, which are common in genomics pipelines. When applications exhibit bursty parallelism or phase shifts, GENOX’s thread-based scaler reacts immediately, avoiding the lag inherent in utilization-driven policies. Its fine-grained monitor also prevents idle bursts from being misinterpreted as reduced demand, ensuring quotas remain aligned with true compute needs.

These capabilities are particularly advantageous in multi-threaded genomics applications (*e.g.*, BWA, STAR, GATK), which naturally progress through phases of fluctuating demand. In production pipelines, stages like alignment, sorting, and indexing exhibit diverse compute and I/O requirements, and GENOX can sustain throughput by adapting quotas without manual tuning. Under these conditions, GENOX consistently outperforms existing baselines, delivering faster application completion times and shorter end-to-end workflows.

**Limitations of GenoX.** While effective in many settings, GENOX can provide limited benefits under certain conditions.

First, GENOX’s reliance on thread count as its primary scaling signal offers *limited ad-*

*advantages for single-threaded applications.* In these cases, execution is inherently serial, and GENOX simply allocates one core throughout the run, performing no worse than Linux defaults but offering no additional speedup. To address this limitation, we will explore incorporating additional scaling signals into GENOX. The Linux kernel exposes scheduler statistics through ‘/sys/fs/’ and ‘/proc/’ interfaces that could enhance compute demand estimation. While some metrics remain accessible through these standard interfaces, preserving GENOX’s non-invasive design, others would require kernel integration. This presents a tradeoff between maintaining a non-invasive approach and achieving more sophisticated scaling capabilities.

Second, GENOX provides *minimal benefits in low-contention environments* where resources are abundant and workflows run in isolation. To quantify this limitation, we compared GENOX against Linux’s greedy (NoLimit) policy, which allows applications to use all available cores without restriction. We executed DNA-Seq and RNA-Seq workflows without contention and measured their total execution time. As shown in Table 4.5, GENOX was only about 1% slower than NoLimit in these resource-rich scenarios. This result confirms that while GENOX’s thread monitoring and quota enforcement introduce negligible overhead, they provide no advantage when contention is absent. This limitation is expected since GENOX is designed for busy production environments, where multiple workflows compete for resources, not for isolated executions with a sufficient number of cores.

## 4.6 Conclusion

We presented GENOX, a dynamic CPU quota allocator for genomics workflows in shared clusters. Through in-depth characterization of genomics workflows, we identified three key requirements for CPU quota management: 1) responsiveness to thread bursts, 2) robustness under contention, and 3) adaptiveness to workflow dynamics. GENOX addresses these requirements using thread counts as its primary scaling signal, operating through three com-

ponents: a thread-based scaler, a 5ms-interval monitor, and a container-native controller via standard cgroup interfaces. Evaluation on DNA-Seq and RNA-Seq workflows showed that GENOX successfully reduced execution time by up to 50%, minimized straggler slowdowns, and improved end-to-end workflow execution time by over 20% compared to state-of-the-art baselines, confirming that thread-based scaling effectively addresses the unique CPU allocation challenges of genomics workloads.

## CHAPTER 5

### RELATED WORK

#### 5.1 Automated Translation

Several works employed learning techniques to achieve automatic translation between general-purpose programming and scripting languages. NGST2 [64] developed a neural architecture capable of translating sections of imperative general-purpose languages into their functional language counterparts, supporting convenient parallelization. DuoGlot [44] addresses the challenge of maintaining readable automatic translations, even when the source code has different coding styles, by incorporating user feedback. In contrast to these works, CNT does not rely on learning techniques, avoiding the need for extensive datasets which are rare yet necessary to train models effectively.

As efficiency and safety are both crucial for systems-level applications, there are several attempts to translate legacy code to memory-safe Rust. Emre *et al.* developed techniques to automatically generate safe Rust code from unsafe segments of C code [65]. Lunnikivi *et al.* semi-automatically translated Python code into low-level code for efficiency using Rust as intermediate representation [66]. These projects have a similarity to CNT as they perform automated translation followed by a manual refinement. However, CNT’s primary focus lies in targeting domain-specific languages, rather than the broader general-purpose languages such as C and Python.

With the increasing use of accelerator hardware, various research domains, including bioinformatics, have increasingly adopted GPUs to handle their ever-expanding workloads [67]. To facilitate GPU support, there are attempts to translate legacy code into CUDA. GPSME [68] offers semi-automatic source-to-source translation of SME applications originally written in C and C++ to CUDA. Conversely, MocCUDA [69] automatically translates CUDA code into OpenMP to take advantage of CPU-only supercomputers.

## 5.2 CPU Quota Allocation

CPU quota allocation for parallel applications has been extensively studied across different domains. We categorize prior work based on their allocation strategies and compare them with GENOX. Table 5.1 summarizes this comparison.

**Utilization-based Reactive Approaches.** Autopilot, AutoThrottle, and SHOWAR share our non-invasive cgroup-based approach but differ fundamentally in their scaling signals. As demonstrated in §4.4, their *reactive nature* prevents them from handling the inflatable parallelism and millisecond-scale idle bursts (§4.2.1) characteristic of genomics workloads.

**Predictive Approaches.** Nodens [12], SmartHarvest [13], and Libra [14] use prediction models to estimate future demand. Nodens combines traffic monitoring with prediction models for bandwidth control. SmartHarvest and Libra employ machine learning for demand forecasting. While proactive, they require training data and historical patterns. For genomics workflows with input-dependent performance variability (§4.2.1) and dynamic branching (§4.2.3), prediction accuracy becomes challenging, potentially leading to misallocation.

**Application-Integrated Approaches.** Arachne [70], Shenango [71], and Caladan [72] achieve microsecond-scale responsiveness through application integration. Arachne requires rewriting applications with its core-aware threading API. Shenango and Caladan use runtime libraries while maintaining POSIX compatibility to minimize code changes. Caladan uniquely employs network queue depths as demand signals. These systems target latency-critical microservices with microsecond requirements, three orders of magnitude finer than genomics applications’ millisecond-scale phases. Moreover, they require code modifications or runtime linking, both impractical for workflows using dozens of unmodified third-party tools.

**Kernel/Hypervisor-based Approaches.** HarvestVM [15] and PerfIso [16] operate through system software modifications. HarvestVM works at the hypervisor layer to harvest idle re-

Table 5.1: **GenoX compared with prior CPU allocation systems for genomics workloads.** *In the context of genomics workload execution, existing systems achieve only a subset of properties, while GENOX uniquely provides responsiveness, robustness, and adaptiveness.*

| System            | Responsiveness | Robustness | Adaptiveness |
|-------------------|----------------|------------|--------------|
| Autopilot [9]     | .              | .          | ✓            |
| Autothrottle [10] | .              | .          | ✓            |
| Arachne [70]      | ✓              | ✓          | .            |
| Caladan [72]      | .              | ✓          | .            |
| HarvestVM [15]    | ✓              | .          | .            |
| Libra [14]        | ✓              | .          | ✓            |
| Nodens [12]       | .              | .          | ✓            |
| PerfIso [16]      | ✓              | .          | .            |
| Shenango [71]     | ✓              | ✓          | .            |
| SHOWAR [11]       | .              | .          | ✓            |
| SmartHarvest [13] | ✓              | .          | .            |
| <b>GenoX</b>      | ✓              | ✓          | ✓            |

sources for best-effort workloads. PerfIso implements performance isolation through kernel-level core pinning operations that take tens of milliseconds, which is too coarse-grained for genomics applications’ rapid phase changes. Both systems require privileged access, which is typically restricted in shared production clusters.

**Key differences.** GENOX distinguishes itself from existing approaches through three key differences. First, it uses thread counts instead of utilization or predictions, providing immediate, stall-agnostic demand signals. Second, it employs 5ms monitoring that captures millisecond-scale phase changes without microsecond-level overhead. Finally, it requires no modifications to applications, runtimes, or kernels, using only standard cgroup interfaces for production deployment.

Table 5.1 demonstrates that GENOX uniquely achieves all three design requirements – *responsiveness*, *robustness*, and *adaptiveness* – in a single production-ready system, making it particularly suited for containerized genomics workflows.

## CHAPTER 6

### FUTURE WORKS

There are many areas in which genomics data processing may benefit from advances in systems research. In this section, we outline several directions for further strengthening genomics data processing systems. These directions span multiple facets of systems, ranging from workflow languages, resource management, to scheduling. We also discuss how deeper understanding of domain-specific characteristics may affect systems design.

#### 6.1 Meta-Compiler for Language-Agnostic Workflows

As mentioned in §3.1, the bioinformatics community are using a variety of workflow engines, where each engine has its own unique strengths and respective workflow language. As a result, many state-of-art genomics workflows developed by panel of experts are difficult to be used and extended outside the research group where they are developed. Simply running the workflows requires setting up an environment/infrastructure that uses specific workflow engine, which takes time and engineering expertise. Extending the workflows also requires language expertise that are traditionally not needed in the researcher’s own group. Moreover, this makes it difficult for systems researchers to assess the performance of state-of-art workflows across different workflow engines and to evaluate workflow engines strengths.

This problem can be solved by creating an abstraction of workflows that preserve correctness, performance, and readability across languages. A meta-compiler uses the abstraction to provide intermediate states for translating workflows from any source to any target language. It then converts the intermediate states into an almost complete, if not complete, workflow code automatically. Furthermore, the compiler should be able to do optimizations according to the target workflow engine, such as using efficient dataflow paradigm when translating to Nextflow. Such meta-compiler will free research groups to run any work-

flows and use any workflow engines without restrictions on time and expertise, and greatly accelerate reproducibility across research groups.

## 6.2 Cloud-Agnostic Execution

Many genomics data processing systems use a hybrid-cloud model which combines on-premise and public cloud environment for large-scale execution. At the cloud level, many vendors provide their own optimizations on workflow engines and genomics applications. For example, Microsoft Azure provides its own implementation of Cromwell engine, along with BWA and GATK applications [73]. Thus, genomics data processing systems can achieve performance benefits by executing a workflow on public cloud which is specifically optimized for it. This can be achieved by extending the meta-compiler described in §6.1. First, it is important to understand in detail the specific optimizations offered by each cloud vendor on workflow engines and genomics applications. Then, given a workflow, the meta-scheduler predicts the cloud which can provides the highest performance advantage, uses meta-compiler to translate the workflow to use the cloud’s optimized workflow engine, and finally execute the workflow on said cloud. Since workflows often consists of smaller workflows (subworkflows), the meta-scheduler can also be implemented to use multiple clouds in a single workflow execution, sending each subworkflow to a different cloud.

## 6.3 Minimizing Memory Waste in Containerized Execution

In many production systems, containerized execution of genomics applications have become the norm [3]. Containerized execution provides portability and reproducibility that are much needed for genomics data analysis. When large-scale genomics data processing systems uses a workflow engine that supports task-level parallelism, such as Nextflow, multiple application containers often run concurrently on a single machine. This happens when the workflow



DAG contain parallelizable steps, or if the user executes multiple workflows at the same time. However, we observe that this scenario can quickly deplete the host machine’s memory because genomics applications use large amounts of RAM. For example, each instance of STAR for alignment uses at least  $\sim 30$ GB of RAM, while BWA’s RAM usage scales linearly with input.

One optimization that we can do is to minimize memory waste due to duplicated reference files. We observe that, in containerized execution, each application instance will load its own reference files, where the total size can easily reach tens of GBs. While some applications support reusing reference files across executions, such as STAR, this is not applicable in containerized setup because containers are destroyed after execution. Instead, we can build a memory management system that enables sharing across containers. The system will sit between workflow engine and the OS, providing mapping or reuse of memory pages across different container processes. This allows multiple instances of containerized application to use only one set of reference files, effectively removing waste due to duplication.

## 6.4 Reliable and Cost-Efficient Execution on Cloud

Cloud is often used in addition to on-premise cluster to meet growing genomics workload demand. It provides a diverse offerings of machines with various degree of availability and price. Specifically, preemptible instances (e.g., AWS spot) are significantly cheaper than typical on-demand instances [74]. However, preemptible instances do not offer the same level of availability as on-demand instances, because cloud providers can revoke these instances anytime with short notice [75, 76]. Thus, currently, workflow execution on preemptible instances is unreliable and prone to failure because instances may be revoked while execution is ongoing. It is even more the case for workflows with long-running applications. Is it possible to have both reliable, yet cost-efficient execution of genomics workflows using preemptible instances? One way to achieve this is by using an accurate predictor of application-level

execution time and preemptible instances lifetime. Then, we can build a scheduler which takes these predictions and decide with certain level of confidence whether each workflow step is safe to be executed on preemptible instance, thus achieving both cost-efficiency and certain degree of reliability.

## 6.5 Leveraging Domain-specific Characteristics

Finally, our discussion with domain experts reveal that execution time of genomics application might be affected by some domain-specific metrics which are not yet properly studied/documented. For example, one shared anecdote is that while input size is known to have the highest correlation to application execution time, executions using inputs with similar size can take vastly different time given different input "quality". This is also one of the reason why we use inputs with different characteristics in evaluation §4.4.6. We believe the large diversity of how genomics input files are prepared, along with various domain-specific metrics used to describe the input files, might hold information that can be used to design better scheduling and resource management techniques.

## CHAPTER 7

### CONCLUSION

This dissertation addresses the issue of increasing CPU usage in genomics data processing systems. As the rate of genomics data creation increases, systems for processing genomics data also increase in scale and complexity. In particular, these systems require a tremendous amount of computational resources to operate – a production system can process hundreds of thousands of compute-intensive workflows annually and consume a total of tens of millions of core hours. It is necessary to maintain high CPU utilization in order to ensure optimal system throughput.

We propose two solutions to increase CPU usage in genomics data processing systems. Our first solution, **CNT**, helps increase CPU usage by facilitating migration to a workflow engine with a high degree of task parallelism. Specifically, **CNT** enables semi-automatic translation of CWL workflows into Nextflow. This allows us to run legacy yet widely-used production-grade workflows written in CWL using Nextflow engine, which natively supports advanced parallelism. **CNT** achieves high translation coverage, thus significantly alleviating the work needed to manually translate workflows. Moreover, Nextflow workflows written using **CNT** achieve higher CPU utilization, and subsequently faster execution times, when compared to CWL workflows run on cwltool.

Our second solution, **GENOX**, increases CPU usage through efficient CPU quota allocation. **GENOX** is designed based on insights that we gained during an in-depth characterization study of widely used genomics workflows. As the study reveals unique challenges of performance variability, contention-induced slowdown, and shifting demand, **GENOX** dynamically manages CPU allocation in a responsive, robust, and adaptive manner. It uses thread signals along with fine-grained monitoring to detect true demand and allocate appropriate CPU quota. Across multiple microbenchmarks, **GENOX** sustains higher CPU utilization compared to state-of-arts CPU allocation approaches, while these approaches ex-

perience performance penalties due to delays in allocation. GENOX also reduces end-to-end execution times of workflows and minimizes straggler slowdown under contention.

In summary, this dissertation provides practical ways to increase CPU usage and ensure optimal systems throughput through two means: increased task parallelism and efficient CPU allocation. It paves the way for further strengthening the resource management aspect of genomics processing systems.

## REFERENCES

- [1] Michael Hao Tong, Robert L. Grossman, and Haryadi S. Gunawi. Experiences in Managing the Performance and Reliability of a Large-Scale Genomics Cloud Platform. In *2021 USENIX Annual Technical Conference (USENIX ATC)*, July 14-16, 2021.
- [2] Common Workflow Language Standards, v1.2. <https://www.commonwl.org/v1.2/>.
- [3] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame. Nextflow Enables Reproducible Computational Workflows. *Nature Biotechnology*, 2017.
- [4] Overview of GDC Harmonization Workflows. <https://github.com/NCI-GDC/gdc-workflow-overview>.
- [5] Z. Zhang, K. Hernandez, J. Savage, S. Li, D. Miller, S. Agrawal, F. Ortuno, L. M. Staudt, A. Heath, and R. L. Grossman. Uniform genomic data analysis in the NCI Genomic Data Commons. *Nature Communications*, 2021.
- [6] cwltool: The reference implementation of the common workflow language standards. <https://github.com/common-workflow-language/cwltool>.
- [7] Jonathan Bader, Fabian Lehmann, Alexander Groth, Lauritz Thamsen, Dominik Scheinert, Jonathan Will, Ulf Leser, and Odej Kao. Reshi: Recommending resources for scientific workflow tasks on heterogeneous infrastructures. In *2022 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pages 269–274, 2022.
- [8] Jonathan Bader, Nicolas Zunker, Soeren Becker, and Odej Kao. Leveraging reinforcement learning for task resource allocation in scientific workflows. In *2022 IEEE International Conference on Big Data (Big Data)*, pages 3714–3719, 2022.
- [9] Krzysztof Rządca, Paweł Findeisen, Jacek Swiderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmirek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload Autoscaling at Google. In *Fifteenth EuroSys Conference (EuroSys)*, April, 2020.
- [10] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y. Yan. Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, April, 2024.
- [11] Ataollah Fatahi Baarzi and George Kesidis. SHOWAR: Right-Sizing And Efficient Scheduling of Microservices. In *ACM Symposium on Cloud Computing (SoCC)*, November, 2021.

- [12] Jiuchen Shi, Hang Zhang, Zhixin Tong, Quan Chen, Kaihua Fu, and Minyi Guo. Nodens: Enabling Resource Efficient and Fast QoS Recovery of Dynamic Microservice Applications in Datacenters. In *USENIX Annual Technical Conference (2023)*, July, 2023.
- [13] Yawen Wang, Kapil Arya, Marios Kogias, Manohar Vanga, Aditya Bhandari, Neeraja J. Yadwadkar, Siddhartha Sen, Sameh Elnikety, Christos Kozyrakis, and Ricardo Bianchini. SmartHarvest: Harvesting Idle CPUs Safely and Efficiently in the Cloud. In *Sixteenth European Conference on Computer Systems (EuroSys)*, April, 2021.
- [14] Hanfei Yu, Christian Fontenot, Hao Wang, Jian Li, Xu Yuan, and Seung-Jong Park. Libra: Harvesting Idle Resources Safely and Timely in Serverless Clusters. In *The 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, June, 2023.
- [15] Pradeep Ambati, Iñigo Goiri, Felipe Vieira Frujeri, Alper Gun, Ke Wang, Brian Dolan, Brian Corell, Sekhar Pasupuleti, Thomas Moscibroda, Sameh Elnikety, Marcus Fountoura, and Ricardo Bianchini. Providing SLOs for Resource-Harvesting VMs in Cloud Platforms. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, November, 2020.
- [16] Calin Iorgulescu, Reza Azimi, Youngjin Kwon, Sameh Elnikety, Manoj Syamala, Vivek R. Narasayya, Herodotos Herodotou, Paulo Tomita, Alex Chen, Jack Zhang, and Junhua Wang. PerfIso: Performance Isolation for Commercial Latency-Sensitive Services. In *USENIX Annual Technical Conference (ATC)*, July, 2018.
- [17] Nextflow. <https://www.nextflow.io/docs/latest/>.
- [18] Common Workflow Language User Guide. [https://www.commonwl.org/user\\_guide/](https://www.commonwl.org/user_guide/).
- [19] Cromwell. <https://cromwell.readthedocs.io/en/latest/>.
- [20] Martin L. Putra, In Kee Kim, Haryadi S. Gunawi, and Robert L. Grossman. CNT: Semi-Automatic Translation from CWL to Nextflow for Genomic Workflows. In *The 23rd IEEE International Conference on Bioinformatics and Bioengineering (BIBE)*, Virtual, November, 2023.
- [21] Oleksandr Rudyy, Marta Garcia-Gasulla, Filippo Mantovani, Alfonso Santiago, Raül Sirvent, and Mariano Vázquez. Containers in HPC: A Scalability and Portability Study in Production Biological Simulations. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May, 2019.
- [22] Azza E. Ahmed, Joshua M. Allen, Tajesvi Bhat, Prakruthi Burra, Christina E. Fliege, Steven N. Hart, Jacob R. Heldenbrand, Matthew E. Hudson, Dave Deandre Istanto, Michael T. Kalmbach, Gregory D. Kapraun, Katherine I. Kendig, Matthew Charles Kendzior, Eric W. Klee, Nate Mattson, Christian A. Ross, Sami M. Sharif, Ramshankar

- Venkatakrishnan, Faisal M. Fadlelmola, and Liudmila S. Mainzer. Design Considerations for Workflow Management Systems Use in Production Genomics Research and the Clinic. *Scientific Reports*, 11(1):21680, 2021.
- [23] Michael R. Crusoe, Sanne Abeln, Alexandru Iosup, Peter Amstutz, John Chilton, Nebojša a Tijanić, Hervé Ménager, Stian Soiland-Reyes, Bogdan Gavrilović, Carole Goble, and The CWL Community. Methods included. *Communications of the ACM*, 2022.
- [24] OpenWDL - Community driven open-development workflow language. <https://openwdl.org>.
- [25] Stephen F. Altschul, Warren Gish, Webb Miller, Eugene W. Myers, and David J. Lipman. Basic Local Alignment Search Tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.
- [26] Stepanka Zverinova and Victor Guryev. Variant Calling: Considerations, Practices, and Developments. *Human Mutation*, 43(8):976–985, 2022.
- [27] David C. Samuels, Hui Yu, and Yan Guo. Is It Time to Reassess Variant Annotation? *Trends in Genetics*, 38(6):521–523, 2022.
- [28] Taylor Reiter, Phillip T Brooks, Luiz Irber, Shannon E K Joslin, Charles M Reid, Camille Scott, C Titus Brown, and N Tessa Pierce-Ward. Streamlining Data-Intensive Biology with Workflow Systems. *GigaScience*, 10(1):giaa140, 2021.
- [29] Moiz Arif, Avinash Maurya, M. Mustafa Rafique, Dimitrios S. Nikolopoulos, and Ali Raza Butt. Application-Attuned Memory Management for Containerized HPC Workflows. In *IEEE International Parallel and Distributed Processing Symposium*, May, 2024.
- [30] Sergey Nurk, Sergey Koren, Arang Rhie, Mikko Rautiainen, Andrey V. Bzikadze, Alla Mikheenko, Mitchell R. Vollger, Nicolas Altemose, Lev Uralsky, Ariel Gershman, Sergey Aganezov, Savannah J. Hoyt, Mark Diekhans, Glennis A. Logsdon, Michael Alonge, Stylianos E. Antonarakis, Matthew Borchers, Gerard G. Bouffard, Shelise Y. Brooks, Gina V. Caldas, Nae-Chyun Chen, Haoyu Cheng, Chen-Shan Chin, William Chow, Leonardo G. de Lima, Philip C. Dishuck, Richard Durbin, Tatiana Dvorkina, Ian T. Fiddes, Giulio Formenti, Robert S. Fulton, Arkarachai Functammasan, Erik Garrison, Patrick G. S. Grady, Tina A. Graves-Lindsay, Ira M. Hall, Nancy F. Hansen, Gabrielle A. Hartley, Marina Haukness, Kerstin Howe, Michael W. Hunkapiller, Chirag Jain, Miten Jain, Erich D. Jarvis, Peter Kerpedjiev, Melanie Kirsche, Mikhail Kolmogorov, Jonas Korlach, Milinn Kremitzki, Heng Li, Valerie V. Maduro, Tobias Marschall, Ann M. McCartney, Jennifer McDaniel, Danny E. Miller, James C. Mullikin, Eugene W. Myers, Nathan D. Olson, Benedict Paten, Paul Peluso, Pavel A. Pevzner, David Porubsky, Tamara Potapova, Evgeny I. Rogaev, Jeffrey A. Rosenfeld, Steven L. Salzberg, Valerie A. Schneider, Fritz J. Sedlazeck, Kishwar Shafin, Colin J.

- Shew, Alaina Shumate, Ying Sims, Arian F. A. Smit, Daniela C. Soto, Ivan Sović, Jessica M. Storer, Aaron Streets, Beth A. Sullivan, Françoise Thibaud-Nissen, James Torrance, Justin Wagner, Brian P. Walenz, Aaron Wenger, Jonathan M. D. Wood, Chunlin Xiao, Stephanie M. Yan, Alice C. Young, Samantha Zarate, Urvashi Surti, Rajiv C. McCoy, Megan Y. Dennis, Ivan A. Alexandrov, Jennifer L. Gerton, Rachel J. O'Neill, Winston Timp, Justin M. Zook, Michael C. Schatz, Evan E. Eichler, Karen H. Miga, and Adam M. Phillippy. The Complete Sequence of a Human Genome. *Science*, 376(6588):44–53, 2022.
- [31] Mohsina Patwekar, Faheem Patwekar, A Venkata Badarinath, Abdul Ajeed Mo-hathasim Billah, Vamseekrishna Gorijavolu, Karthickeyan Krishnan, Palani Shanmu-gasundaram, P Dharani Prasad, and A A Kazi. Genomic sequencing: Techniques, advancements, and the path ahead. *J. BioX Res.*, 8, January 2025.
- [32] Christopher Huptas, Siegfried Scherer, and Mareike Wenning. Optimized illumina PCR-free library preparation for bacterial whole genome sequencing and analysis of factors influencing de novo assembly. *BMC Res. Notes*, 9(1):269, May 2016.
- [33] Dong H. Ahn, Xiaohua Zhang, Jeffrey Mast, Stephen Herbein, Francesco Di Natale, Dan Kirshner, Sam Ade Jacobs, Ian Karlin, Daniel J. Milroy, Bronis R. de Supinski, Brian Van Essen, Jonathan E. Allen, and Felice C. Lightstone. Scalable Composi-tion and Analysis Techniques for Massive Scientific Workflows. In *IEEE International Conference on e-Science (e-Science)*, October, 2022.
- [34] Xiangqun Zheng-Bradley, Ian Streeter, Susan Fairley, David Richardson, Laura Clarke, Paul Flicek, and the 1000 Genomes Project Consortium. Alignment of 1000 Genomes Project Reads to Reference Assembly GRCh38. *GigaScience*, 6(7):gix038, 05 2017.
- [35] Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple Linux Utility for Resource Management. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, June, 2003.
- [36] Robert L. Henderson. Job Scheduling Under the Portable Batch System. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, April, 1995.
- [37] Garrick Staples. TORQUE - TORQUE resource manager. In *ACM/IEEE Conference on High Performance Networking and Computing (SC)*, November, 2006.
- [38] Kubernetes Horizontal Pod Autoscaling. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>.
- [39] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. Apache Hadoop YARN: Yet Another Resource Negotiator. In *ACM Symposium on Cloud Computing (SoCC)*, November, 2013.



- [40] L. Wratten, A. Wilm, and J. ke. Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers. *Nature Methods*, 2021.
- [41] Allison P Heath, Vincent Ferretti, Stuti Agrawal, Maksim An, James C Angelakos, Renuka Arya, Rosita Bajari, Bilal Baqar, Justin HB Barnowski, Jeffrey Burt, and et al. The NCI Genomic Data Commons. *Nature Genetics*, 2021.
- [42] cwl-utils. <https://github.com/common-workflow-language/cwl-utils>.
- [43] Nextflow Patterns. <https://nextflow-io.github.io/patterns/optional-input/>.
- [44] Bo Wang, Aashish Kolluri, Ivica Nikolić, Teodora Baluta, and Prateek Saxena. User-Customizable Transpilation of Scripting Languages. *Proceedings of ACM Programming Language*, 2023.
- [45] Fabian Lehmann, Jonathan Bader, Friedrich Tschirpke, Lauritz Thamsen, and Ulf Leser. How Workflow Engines Should Talk to Resource Managers: A Proposal for a Common Workflow Scheduling Interface. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, May, 2023.
- [46] Md. Vasimuddin, Sanchit Misra, Heng Li, and Srinivas Aluru. Efficient Architecture-Aware Acceleration of BWA-MEM for Multicore Systems. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May, 2019.
- [47] Sergei Yakneen, Sebastian M. Waszak, Michael Gertz, and Jan O. Korbel. Butler Enables Rapid Cloud-Based Analysis of Thousands of Human Genomes. *Nature Biotechnology*, 38:288 – 292, 2020.
- [48] Alexander Dobin, Carrie A. Davis, Felix Schlesinger, Jorg Drenkow, Chris Zaleski, Sonali Jha, Philippe Batut, Mark Chaisson, and Thomas R. Gingeras. STAR: Ultrafast Universal RNA-Seq Aligner. *Bioinformatics*, 29(1):15–21, 10 2012.
- [49] Genome Analysis Toolkit. <https://gatk.broadinstitute.org/hc/en-us>.
- [50] T. M. A. Do, Loïc Pottier, R. Ferreira da Silva, F. Suter, S. Caíno-Lores, M. Taufer, E. Deelman, and T. Peterka. Accelerating Scientific Workflows on HPC Platforms with In Situ Processing. In *IEEE/ACM 22nd International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, May, 2022.
- [51] FastQC. <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>.
- [52] Anthony M. Bolger, Marc Lohse, and Bjoern Usadel. Trimmomatic: a Flexible Trimmer for Illumina Sequence Data. *Bioinformatics*, 30(15):2114–2120, 2014.
- [53] Jonathan Bader, Joel Witzke, Soeren Becker, Ansgar Löffler, Fabian Lehmann, Leon Doehler, Anh Duc Vu, and Odej Kao. Towards Advanced Monitoring for Scientific Workflows. In *IEEE International Conference on Big Data (Big Data)*, December, 2022.

- [54] Jonathan Bader, Lauritz Thamsen, Svetlana Kulagina, Jonathan Will, Henning Meyerhenke, and Odej Kao. Tarema: Adaptive Resource Allocation for Scalable Scientific Workflows in Heterogeneous Clusters. In *IEEE International Conference on Big Data (Big Data)*, December, 2021.
- [55] Alok Singh, Shweta Purawat, Arvind Rao, and Ilkay Altintas. Modular Performance Prediction for Scientific Workflows Using Machine Learning. *Future Generation Computing Systems*, 114:1–14, 2021.
- [56] Ewa Deelman, Anirban Mandal, Ming Jiang, and Rizos Sakellariou. The Role of Machine Learning in Scientific Workflows. *International Journal of High Performance Computing Applications*, 33(6), 2019.
- [57] Heng Li and Richard Durbin. Fast and Accurate Short Read Alignment with Burrows–Wheeler Transform. *Bioinformatics*, 25(14):1754–1760, 05 2009.
- [58] German Tischler and Steven Leonard. Biobambam: Tools for Read Pair Collation-Based Algorithms on BAM Files. *Source Code for Biology and Medicine*, 9:13, 2014.
- [59] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. Lessons Learned from the Chameleon Testbed. In *USENIX Annual Technical Conference (ATC)*, July, 2020.
- [60] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. TMO: Transparent Memory Offloading in Datacenters. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, February, 2022.
- [61] GATK - CollectOxoGMetrics (Picard). <https://gatk.broadinstitute.org/hc/en-us/articles/360037227552-CollectOxoGMetrics-Picard>.
- [62] Rasmus Amund Henriksen, Lei Zhao, and Thorfinn Sand Korneliussen. Ngsngs: next-generation simulator for next-generation sequencing data. *Bioinformatics*, 39(1), January 2023.
- [63] National Library of Medicine - Sequence Read Archive (SRA). <https://www.ncbi.nlm.nih.gov/sra>.
- [64] Benjamin Mariano, Yanju Chen, Yu Feng, Greg Durrett, and Isil Dillig. Automated transpilation of imperative to functional code using neural-guided program synthesis. *Proceedings of the ACM on Programming Languages*, 2022.
- [65] Mehmet Emre, Ryan Schroeder, Kyle Dewey, and Ben Hardekopf. Translating C to Safer Rust. *Proceedings of ACM Programming Language*, 2021.

- [66] Henri Lunnikivi, Kai Jylkkä, and Timo Hämäläinen. Transpiling Python to Rust for Optimized Performance. In *International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*, 2020.
- [67] A. Taylor-Weiner, F. Aguet, N. J. Haradhvala, S. Gosai, S. Anand, J. Kim, K. Ardlie, E. M. Van Allen, and G. Getz. Scaling computational genomics to millions of individuals with GPUs. *Genome Biology*, 2019.
- [68] Po Yang and *et al.* Improving Utility of GPU in Accelerating Industrial Applications With User-Centered Automatic Code Translation. *IEEE Transactions on Industrial Informatics*, 2018.
- [69] William S. Moses, Ivan R. Ivanov, Jens Domke, Toshio Endo, Johannes Doerfert, and Oleksandr Zinenko. High-Performance GPU-to-CPU Transpilation and Optimization via High-Level Parallel Constructs. In *ACM SIGPLAN Symposium on Principles & Practice of Parallel Programming (PPoPP)*, 2023.
- [70] Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John K. Ousterhout. Arachne: Core-Aware Thread Management. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October, 2018.
- [71] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, February, 2019.
- [72] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating Interference at Microsecond Timescales. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, November, 2020.
- [73] Accelerating Genomics Workflows and Data Analysis on Azure. <https://azure.microsoft.com/en-us/blog/accelerating-genomics-workflows-and-data-analysis-on-azure/>.
- [74] Amazon Web Services. Amazon EC2 On-Demand Pricing. <https://aws.amazon.com/ec2/pricing/on-demand/>, 2026.
- [75] Pedro Joaquim, Manuel Bravo, Luís Rodrigues, and Miguel Matos. Hourglass: Leveraging transient resources for time-constrained graph processing in the cloud. In *Proceedings of the Fourteenth EuroSys Conference 2019*, EuroSys '19, New York, NY, USA, 2019. Association for Computing Machinery.
- [76] Supreeth Shastri and David E. Irwin. Hotspot: automated server hopping in cloud spot markets. In *Proceedings of the 2017 Symposium on Cloud Computing, SoCC 2017, Santa Clara, CA, USA, September 24-27, 2017*, pages 493–505. ACM, 2017.
- [77] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

- [78] Chameleon: A configurable experimental environment for large-scale edge to cloud research. <https://www.chameleoncloud.org/>.
- [79] NVIDIA Clara Parabricks v4.0.1. <https://docs.nvidia.com/clara/parabricks/>.
- [80] Kyriakos-Ioannis D. Kyriakou, Ioannis K. Chaniotis, and Nikolaos D. Tselikas. The gpm meta-transcompiler: Harmonizing javascript-oriented web development with the upcoming ecma-script 6 “harmony” specification. In *2015 12th Annual IEEE Consumer Communications and Networking Conference (CCNC)*, pages 176–181, 2015.
- [81] Yadu Babuji, Anna Woodard, Zhuozhao Li, Daniel S. Katz, Ben Clifford, Rohan Kumar, Lukasz Lacinski, Ryan Chard, Justin M. Wozniak, Ian Foster, Michael Wilde, and Kyle Chard. Parsl: Pervasive parallel programming in python. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, HPDC ’19, page 25–36, New York, NY, USA, 2019. Association for Computing Machinery.
- [82] Burkhard Ringlein, Francois Abel, Alexander Ditter, Beat Weiss, Christoph Hagleitner, and Dietmar Fey. Programming reconfigurable heterogeneous computing clusters using mpi with transpilation. In *2020 IEEE/ACM International Workshop on Heterogeneous High-performance Reconfigurable Computing (H2RC)*, pages 1–9, 2020.
- [83] Miguel Marcelino and António Menezes Leitão. Extending pyjl - transpiling python libraries to julia. In João Cordeiro, Maria João Pereira, Nuno F. Rodrigues, and Sebastião Pais, editors, *11th Symposium on Languages, Applications and Technologies, SLATE 2022, July 14-15, 2022, Universidade da Beira Interior, Covilhã, Portugal*, volume 104 of *OASICs*, pages 6:1–6:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
- [84] Qian Zhang, Jiyuan Wang, Guoqing Harry Xu, and Miryung Kim. Heterogen: Transpiling c to heterogeneous hls code with automated test generation and program repair. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’22, page 1017–1029, New York, NY, USA, 2022. Association for Computing Machinery.
- [85] Subroto Nag Pinku, Debajyoti Mondal, and Chanchal K. Roy. Pathways to leverage transcompiler based data augmentation for cross-language clone detection, 2023.
- [86] Emily Bourne, Yaman Güçlü, Said Hadjout, and Ahmed Ratnani. Pyccel: a python-to-x transpiler for scientific high-performance computing. *Journal of Open Source Software*, 8(83):4991, 2023.
- [87] Linyuan Gong, Jiayi Wang, and Alvin Cheung. Adelt: Transpilation between deep learning frameworks, 2023.
- [88] C. S. Greene, J. Tan, M. Ung, J. H. Moore, and C. Cheng. Big Data Bioinformatics. *Journal of Cellular Physiology*, 229(12):1896–1900, Dec 2014.

- [89] Z. D. Stephens, S. Y. Lee, F. Faghri, R. H. Campbell, C. Zhai, M. J. Efron, R. Iyer, M. C. Schatz, S. Sinha, and G. E. Robinson. Big Data: Astronomical or Genomical? *PLOS Biology*, 13(7):e1002195, Jul 2015.
- [90] Google Cloud. Google Cloud Compute Engine Pricing. <https://cloud.google.com/compute/all-pricing>, 2026.
- [91] Neeraja J. Yadwadkar, Bharath Hariharan, Joseph E. Gonzalez, Burton Smith, and Randy H. Katz. Selecting the Best VM across Multiple Public Clouds: A Data-Driven Performance Modeling Approach. In *ACM Symposium on Cloud Computing (SoCC)*, 2017.
- [92] Carl Witt, Dennis Wagner, and Ulf Leser. Feedback-based resource allocation for batch scheduling of scientific workflows. In *International Conference on High Performance Computing & Simulation (HPCS)*, 2019.
- [93] Shivaram Venkataraman, Zongheng Yang, Michael J. Franklin, Benjamin Recht, and Ion Stoica. Ernest: Efficient performance prediction for large-scale advanced analytics. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2016.
- [94] Geraldine A. Van der Auwera, Mauricio O. Carneiro, Christopher Hartl, Ryan Poplin, Guillermo del Angel, Ami Levy-Moonshine, Tadeusz Jordan, Khalid Shakir, David Roazen, Joel Thibault, Eric Banks, Kiran V. Garimella, David Altshuler, Stacey Gabriel, and Mark A. DePristo. From FastQ Data to High-Confidence Variant Calls: The Genome Analysis Toolkit Best Practices Pipeline. *Current Protocols in Bioinformatics*, 43:11.10.1–11.10.33, 2013.
- [95] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel Distributed Systems*, 13(3):260–274, 2002.
- [96] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhijing Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. Borg: The Next Generation. In *European Conference on Computer Systems (EuroSys)*, 2020.
- [97] Aidan P. Thompson, Hasan Metin Aktulga, Richard Berger, Dan S. Bolintineanu, W. Michael Brown, Paul S. Crozier, Pieter J. In 't Veld, Axel Kohlmeyer, Stan G. Moore, Trung Dac Nguyen, Ray Shan, Mark J. Stevens, Julien Tranchida, Christian Trott, and Steven J. Plimpton. LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271:108171, 2022.
- [98] Arun Subramaniyan, Yufeng Gu, Timothy Dunn, Somnath Paul, Md. Vasimuddin, Sanchit Misra, David T. Blaauw, Satish Narayanasamy, and Reetuparna Das. GenomicsBench: A Benchmark Suite for Genomics. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2021.

- [99] Allan Snaveley, Laura Carrington, Nicole Wolter, Jesús Labarta, Rosa Badia, and Avi Purkayastha. A Framework for Performance Modeling and Prediction. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2002.
- [100] Michel J. F. Rosa, Célia Ghedini Ralha, Maristela Holanda, and Aletéia P. F. Araújo. Computational resource and cost prediction service for scientific workflows in federated clouds. *Future Generation Computing Systems*, 125:844–858, 2021.
- [101] Seungwoo Rho, Geunchul Park, Jieun Choi, and Chan-Yeol Park. Development of benchmark automation suite and evaluation of various high-performance computing systems. *Cluster Computing*, 24(1), 2021.
- [102] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Basar, and Ravi K. Iyer. FLASH: Fast Model Adaptation in ML-Centric Cloud Platforms. In *The Seventh Annual Conference on Machine Learning and Systems (MLSys)*, 2024.
- [103] Sarunya Pumma, Wu-chun Feng, Phond Phunchongharn, Sylvain Chapeland, and Tiranee Achalakul. A runtime estimation framework for ALICE. *Future Generation Computer Systems*, 72:65–77, 2017.
- [104] Jordan G. Powers, Joseph B. Klemp, William C. Skamarock, et al. The Weather Research and Forecasting Model: Overview, System Efforts, and Future Directions. *Bulletin of the American Meteorological Society*, 98(8):1717–1737, 2017.
- [105] Ryan Poplin, Pi-Chuan Chang, David Alexander, et al. A universal snp and small-indel variant caller using deep neural networks. *Nature Biotechnology*, 36:983–987, 2018.
- [106] Phoronix Media. Phoronix Test Suite: Open-Source Automated Benchmarking, 2026.
- [107] Tuan Pham, Juan J. Durillo, and Thomas Fahringer. Predicting Workflow Task Execution Time in the Cloud Using a Two-Stage Machine Learning Approach. *IEEE Transactions on Cloud Computing*, 8(1):256–268, 2020.
- [108] Simon J. Pennycook, Jason D. Sewall, and Victor W. Lee. A metric for performance portability. *CoRR*, abs/1611.07409, 2016.
- [109] Kyle A. O’Connell, Zelaikha B. Yosufzai, Ross Campbell, Collin J. Lobb, Haley T. Engelken, Laura M. Gorrell, Thad B. Carlson, Josh J. Catana, Dina Mikdadi, Vivien R. Bonazzi, and Juergen A. Klenk. Accelerating genomic workflows using NVIDIA Parabricks. *BMC Bioinformatics*, 24(1):221, 2023.
- [110] Maria Luiza Mondelli, Thiago Magalhães, Guilherme Loss, Michael Wilde, Ian T. Foster, Marta Mattoso, Daniel S. Katz, Helio J. C. Barbosa, Ana Tereza Ribeiro de Vasconcelos, Kary A. C. S. Ocaña, and Luiz M. R. Gadelha Jr. BioWorkbench: A High-Performance Framework for Managing and Analyzing Bioinformatics Experiments. *CoRR*, abs/1801.03915, 2018.



- [111] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twarber, Allen Kanber, Simon Wilber, Matthew Hemberg, Altuna Akalin, Nigel Delaney, Luca Costanzo, and Johannes Köster. Sustainable Data Analysis with Snakemake. *F1000Research*, 10:33, 2021.
- [112] Harshitha Menon, Abhinav Bhatele, and Todd Gamblin. Auto-Tuning Parameter Choices in HPC Applications Using Bayesian Optimization. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2020.
- [113] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning Scheduling Algorithms for Data Processing Clusters. In *ACM SIGCOMM Conference (SIGCOMM)*, 2019.
- [114] Ashraf Mahgoub, Alexander Michaelson Medoff, Rakesh Kumar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. OPTIMUSCLOUD: Heterogeneous Configuration Optimization for Distributed Databases in the Cloud. In *USENIX Annual Technical Conference (USENIX ATC)*, 2020.
- [115] Lorién López-Villellas, Rubén Langarita-Benítez, Asaf Badouh, Víctor Soria Pardos, Quim Aguado-Puig, Guillem López-Paradís, Max Doblas, Javier Setoain, Chulho Kim, Makoto Ono, Adrià Armejach, Santiago Marco-Sola, Jesús Alastruey-Benedé, Pablo Ibáñez, and Miquel Moretó. GenArchBench: A genomics benchmark suite for arm HPC processors. *Future Generation Computer Systems*, 157, 2024.
- [116] Zhuren Liu, Shouzhe Zhang, Justin Garrigus, and Hui Zhao. Genomics-GPU: A Benchmark Suite for GPU-accelerated Genome Analysis. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023.
- [117] Heng Li and Richard Durbin. Fast and accurate long-read alignment with Burrows–Wheeler transform. *Bioinformatics*, 26(5):589–595, 2010.
- [118] Jeremy Leipzig. A Review of Bioinformatic Pipeline Frameworks. *Briefings in Bioinformatics*, 18(3):530–536, 2017.
- [119] Kubernetes Authors. Vertical pod autoscaler, 2026.
- [120] G. Kresse and J. Furthmüller. Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set. *Physical Review B*, 54(16):11169–11186, 1996.
- [121] Ana Klimovic, Heiner Litz, and Christos Kozyrakis. Selecta: Heterogeneous Cloud Storage Configuration for Data Analytics. In *USENIX Annual Technical Conference (ATC)*, 2018.
- [122] Broad Institute. Picard toolkit. <https://broadinstitute.github.io/picard/>, 2019. Broad Institute, GitHub Repository.

- [123] sysstat project. sysstat: System Performance Tools. <https://sysstat.github.io/>, 2026.
- [124] Noah Kaplan, Jan-Niklas Schmelzle, Yufeng Gu, Erik Garrison, Christopher Batten, and Reetuparna Das. Pangenomicsbench: A benchmark suite and characterization of pangenomics. In *IEEE International Symposium on Workload Characterization (IISWC)*, 2025.
- [125] Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2):203–211, 2021.
- [126] Chin-Jung Hsu, Vivek Nair, Vincent W. Freeh, and Tim Menzies. Arrow: Low-Level Augmented Bayesian Optimization for Finding the Best Cloud VM. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2018.
- [127] Torsten Hoefer and Roberto Belli. Scientific Benchmarking of Parallel Computing Systems. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [128] Muhammad Hilman, Maria Rodriguez, and Rajkumar Buyya. Task Runtime Prediction in Scientific Workflows Using an Online Incremental Learning Approach. In *IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CC-Grid)*, 2018.
- [129] Herodotos Herodotou, Fei Dong, and Shivnath Babu. No One (Cluster) Size Fits All: Automatic Cluster Sizing for Data-Intensive Analytics. In *ACM Symposium on Cloud Computing (SOCC)*, 2011.
- [130] Maximilian Hanussek, Felix Bartusch, and Jens Krüger. Performance and scaling behavior of bioinformatic applications in virtualization environments to create awareness for the efficient use of compute resources. *PLoS Computational Biology*, 17(7), 2021.
- [131] Maximilian Hanussek, Felix Bartusch, and Jens Krüger. BOOTABLE: Bioinformatics benchmark tool suite for applications and hardware. *Future Generation Computer Systems*, 102, 2020.
- [132] Wei Gao, Zhisheng Ye, Peng Sun, Yonggang Wen, and Tianwei Zhang. Estimating GPU Memory Consumption of Deep Learning Models. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020.
- [133] Philip A. Ewels, Alexander Peltzer, Sven Fillinger, Harshil Patel, Johannes Alneberg, Andreas Wilm, Maxime Ulysse Garcia, Paolo Di Tommaso, and Sven Nahnsen. The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology*, 38(3):276–278, 2020.



- [134] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuang-Ching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The Design and Operation of CloudLab. In *USENIX Annual Technical Conference (USENIX ATC)*, 2019.
- [135] G. Danabasoglu, J.-F. Lamarque, J. Bacmeister, et al. The Community Earth System Model Version 2 (CESM2). *Journal of Advances in Modeling Earth Systems*, 12(2):e2019MS001916, 2020.
- [136] Rafael Ferreira da Silva, Gideon Juve, Mats Rynge, Ewa Deelman, and Miron Livny. Online Task Resource Consumption Prediction for Scientific Workflows. *Parallel Processing Letters*, 25(03):1541003, 2015.
- [137] Alexandru Calotoiu, Torsten Hoefer, Marius Poke, and Felix Wolf. Using Automated Performance Modeling to Find Scalability Bugs in Complex Codes. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2013.
- [138] Muhammad Bilal, Marco Canini, and Rodrigo Rodrigues. Finding the Right Cloud Configuration for Analytics Clusters. In *ACM Symposium on Cloud Computing (SoCC)*, 2020.
- [139] Amir Bahmani, Ziyi Xing, Vandhana Krishnan, Utsab Ray, Frank Mueller, Amir Alavi, Philip S. Tsao, Michael P. Snyder, and Cuiping Pan. Hummingbird: Efficient performance prediction for executing genomic applications in the cloud. *Bioinformatics*, 37(17):2537–2543, 2021.
- [140] Jonathan Bader, Kathleen West, Sören Becker, Svetlana Kulagina, Fabian Lehmann, Lauritz Thamsen, Henning Meyerhenke, and Odej Kao. Predicting the Performance of Scientific Workflow Tasks for Cluster Resource Management: An Overview of the State of the Art. *CoRR*, abs/2504.20867, 2025.
- [141] Jonathan Bader, Fabian Skalski, Fabian Lehmann, Dominik Scheinert, Jonathan Will, Lauritz Thamsen, and Odej Kao. Sizey: Memory-Efficient Execution of Scientific Workflow Tasks. In *IEEE International Conference on Cluster Computing (CLUSTER)*, 2024.
- [142] Jonathan Bader, Ansgar Löber, Lauritz Thamsen, Björn Scheuermann, and Odej Kao. KS+: Predicting Workflow Task Memory Usage Over Time. In *IEEE International Conference on e-Science (e-Science)*, 2024.
- [143] Jonathan Bader, Fabian Lehmann, Lauritz Thamsen, Ulf Leser, and Odej Kao. Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures. *Future Generation Computer Systems*, 150:171–185, 2024.

- [144] Thais Aparecida Silva Camacho, Vinicius Martins do Rosario, Otavio Oliveira Napoli, and Edson Borin. PB<sup>3</sup>Opt: Profile-Based Biased Bayesian Optimization to Select Computing Clusters on the Cloud. *Concurrency and Computation: Practice and Experience*, 35(18):e7540, 2022.
- [145] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017.
- [146] Kursad Albayraktaroglu, Aamer Jaleel, Xue Wu, Manoj Franklin, Bruce L. Jacob, Chau-Wen Tseng, and Donald Yeung. BioBench: A Benchmark Suite of Bioinformatics Applications. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2005.
- [147] Chin-Jung Hsu, Vivek Nair, Tim Menzies, and Vincent W. Freeh. Scout: An Experienced Guide to Find the Best Cloud Configuration. In *arXiv preprint arXiv:1803.01296*, 2018.
- [148] Kathleen Voss, Geraldine Van der Auwera, and Jeff Gentry. Full-stack genomics pipelining with GATK4 + WDL + Cromwell. *F1000Research*, 6, 2017.
- [149] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. Pegasus: A workflow management system for science automation. *Future Generation Computer Systems*, 46:17–35, 2015.
- [150] Michael Albrecht, Patrick Donnelly, Peter Bui, and Douglas Thain. Makeflow: A portable abstraction for data intensive computing on clusters, clouds, and grids. In *Workshop on Scalable Workflow Enactment Engines and Technologies (SWEET)*, 2012.
- [151] J. C. S. Kadupitiya, Vikram Jadhao, and Prateek Sharma. SciSpot: Scientific Computing On Temporally Constrained Cloud Preemptible VMs. *IEEE Transactions on Parallel Distributed Systems*, 33(12):3575–3588, 2022.
- [152] B. P. Welford. Note on a Method for Calculating Corrected Sums of Squares and Products. *Technometrics*, 4(3):419–420, 1962.